

# UNIX/FORTRAN/ Bourne Shell Script

自習テキスト

花崎 直太

新田 友子





## はじめに

このテキストは水文学を学ぶ学部生、大学院生を対象とした計算機の自習教材です。このテキストを通読すると、読者は水文学のデータ解析やコンピュータシミュレーションを行う際に必要となる UNIX、FORTRAN、Bourne Shell Script に関する基本的な技術を身につけることができます。

このテキストは東京大学生産技術研究所沖研究室で行われていた初心者向けの UNIX 講座の講義録が元になっています。沖研究室は地球水循環システムの研究を行う研究室で、地球水循環のコンピュータシミュレーションやデータ解析で大きな成果を挙げています。この研究室で修士論文や博士論文を書くには、学生は UNIX とプログラミングに精通することが不可欠です。

さて、沖研究室に在籍する学生はほぼ全員、夏学期に沖教授が担当する「Advanced Hydrology」（講義の日本語名はありませんが無理に訳せば高等水文学）という講義に出席します。2001 年頃、この講義には次のような宿題がありました。

Read the following article:

*Xie, P., and P. A. Arkin (1997), Global Precipitation: A 17-Year Monthly Analysis Based on Gauge Observations, Satellite Estimates, and Numerical Model Outputs, Bull. Amer. Meteor. Soc., 78, 2539-2558.*

- Summarize their methodology
- List up advantage/disadvantage
- Compare the data with other data  
(e.g. station observation, global dataset, etc.)
- See the site below for original data

<ftp://ftp.cpc.ncep.noaa.gov/precip/cmap/monthly/>

要約すると、全球の月単位の降水量データ CMAP (CPC Merged Analysis of Precipitation) の作成を報告した Xie and Arkin (1997)を読み、手法や特徴をまとめた上で、実際に彼らのデータを入手して他の降水量データと比較せよ、ということになります。

受講者（多くは沖研究室に入ったばかりの修士 1 年生、博士 1 年生）は、宿題の後半に苦労しました。CMAP は全球を 2.5 度×2.5 度の解像度でカバーする格子データです。地球は経度方向に 360 度、緯度方向に 180 度ありますから、ひと月のデータは  $144 \times 72 = 10368$  地点の数値からなります。このことは Microsoft Excel のスプレッドシートに、144 列 72 行にわたって数値が並んでいることを想像するとよく分かります。ウェブサイトからダウンロードできるのは、こうした数値が、10368 地点、12 カ月分縦に並んだテキストファイルで、 $10368 \times 12 = 124416$  行あります。受講者の多くは Microsoft Excel を使うことはできましたが、当時の Excel は 65536 行のテキストファイルまでしか開くことができず、まずどうやってファイルを開くか、というところから悩まされたのです。幸いにファイルが開けたとしても、次はどうやって図を作るかに苦労しました。CMAP は全球をカバーするデータですから、降水量の地図を描きたくなるものですが、当時の東京大学社会基盤工学科には GIS ソフトウェアの体系だった講義がなく、多くの学生には地図の描き方が分かりませんでした。結論から言うと、沖研究室にあった UNIX

コンピュータを使いこなさない限り、沖教授を満足させるような宿題は提出できなかったのでした。

ここまでの話をまとめると、沖研究室の大学院学生は UNIX に精通する必要があり、また夏学期に行われる沖教授の講義の宿題を提出するために UNIX を利用した CMAP データの解析を早急に習得する必要がありました。後者で必要な技術を挙げると次のようになります。

- UNIX の操作
- FORTRAN<sup>1</sup>を使ったプログラミング
- Generic Mapping Tools (GMT)<sup>2</sup>ソフトウェアを使った地図の作成
- Bourne Shell Script を使った大量のデータを効率的に処理するための技術

当時大学院学生だった筆者は 2002 年～2005 年度の UNIX 講座を担当しており、こうしたニーズに応えるため講義を編成し、講義録を作成しました。その後、多くの人の協力を得て文章や内容を充実させ、講義録をこのテキストに再編しました。このテキストで解説する技術は宿題の提出だけでなく、修士論文や博士論文の研究にもきっと役立つはずです。また、全球水資源モデル H08<sup>3</sup>に関心がある人にも役立つはずです。H08 で使われている技術の基礎は全てこのテキストから学ぶことができるでしょう。

このテキストは「初学者でも飽きずに最後まで続けられること」を優先して構成されています。特に 1 章読み進めるごとに読者が宿題提出への手ごたえを感じられるように配慮したつもりです。ただし、その結果として、説明の順番が前後したり、体系的な説明になっていなかったりする個所があります。また、このテキストは UNIX, FORTRAN, Bourne シェルスクリプトを網羅的に解説することを意図したものではなく、むしろ、市販の教科書には詳しく書かれないことが多いけれども、水文学のデータを扱う時に必須の技術に絞って解説したものといえます。Appendix D に参考文献を挙げましたので、このテキストと並行して読み進めてほしいと思います。

筆者自身の経験から言うと、UNIX を知っているか知らないかで、またプログラミングできるかできないかで、扱えるデータやシミュレーションが全く変わります。このテキストが読者の UNIX やプログラミングに取り組むきっかけになることを、筆者は願ってやみません。

謝辞：第 3 版の編集にあたり、上野博史さんの助力を得ました。ここに記して感謝します。

<sup>1</sup> FORTRAN は地球科学においては今でも非常によく使われている言語です。大気海洋結合モデルをはじめとする地球科学の重要なコンピュータプログラムの多くが FORTRAN で記述されています。

<sup>2</sup> <http://gmt.soest.hawaii.edu/>

<sup>3</sup> Hanasaki et al. 2008a,b (<http://direct.sref.org/1607-7938/hess/2008-12-1007>, <http://direct.sref.org/1607-7938/hess/2008-12-1027>)で発表された全球水資源モデルのこと。地球の自然の水循環と主要な人間の水利利用を同時に計算することのできる水文モデルである。

## 目次

|                                                         |           |
|---------------------------------------------------------|-----------|
| はじめに.....                                               | I         |
| <b>第1章 UNIXの基本操作(1) 基本的なUNIXコマンド.....</b>               | <b>1</b>  |
| 1.1 UNIX環境.....                                         | 1         |
| 1.2 ログインとログアウト.....                                     | 1         |
| 1.3 UNIXコマンド①——ファイルの管理.....                             | 1         |
| 1.4 UNIXコマンド②——ファイルの編集.....                             | 3         |
| 1.5 UNIXコマンド③——その他のコマンド.....                            | 4         |
| 1.6 UNIXコマンド④——困ったときは.....                              | 4         |
| 1.7 パーミッション.....                                        | 5         |
| 宿題.....                                                 | 6         |
| <b>第2章 UNIXの基本操作(2) REDIRECT/PIPE/WILDCARD/AWK.....</b> | <b>7</b>  |
| 2.1 REDIRECT.....                                       | 7         |
| 2.2 PIPE.....                                           | 8         |
| 2.3 WILDCARD.....                                       | 9         |
| 2.4 AWK.....                                            | 9         |
| <b>第3章 BOURNE シェルスクリプト(1) GMT コマンドを組み合わせる.....</b>      | <b>12</b> |
| 3.1 GMT とは.....                                         | 12        |
| 3.2 GMT のシェルスクリプト.....                                  | 13        |
| 3.3 GMT コマンドのオプション.....                                 | 16        |
| 3.4 CPT ファイル.....                                       | 17        |
| 宿題.....                                                 | 17        |
| <b>第4章 グリッドデータの処理(1) FORTRAN プログラムを利用した演算.....</b>      | <b>19</b> |
| 4.1 アスキーとバイナリ.....                                      | 19        |
| 4.2 月単位データの年単位データへの変換.....                              | 20        |
| 4.3 データの差と図化.....                                       | 23        |
| 宿題.....                                                 | 23        |
| <b>第5章 FORTRAN(1) FORTRAN の入出力.....</b>                 | <b>25</b> |
| 5.1 FORTRANソースコードの基本.....                               | 25        |
| 5.2 FORTRANソースコードのコンパイル.....                            | 26        |
| 5.3 FORTRAN のファイルの入出力.....                              | 26        |
| 宿題.....                                                 | 31        |
| <b>第6章 グリッドデータの処理(2) FORTRAN プログラムを使ったマスク処理.....</b>    | <b>32</b> |
| 6.1 この章で利用するデータ.....                                    | 32        |
| 6.2 CMAP のバイナリファイルを操作するためのコマンド.....                     | 33        |

|                                                            |           |
|------------------------------------------------------------|-----------|
| 宿題.....                                                    | 37        |
| <b>第 7 章 FORTRAN (2) FORTRAN の時系列データの入出力.....</b>          | <b>38</b> |
| 7.1 FORTRAN の時系列ファイルの入出力.....                              | 38        |
| 7.2 FORTRAN の SUBROUTINE.....                              | 42        |
| 7.3 MAKE と MAKEFILE.....                                   | 43        |
| 宿題.....                                                    | 44        |
| <b>第 8 章 BOURNE シェルスクリプト (2) DO ループ、FOR ループと IF 文.....</b> | <b>46</b> |
| 8.1 シェルスクリプトの制御文と条件文.....                                  | 46        |
| 8.2 【例 1】FOR ループを使った BOURNE シェルスクリプト.....                  | 47        |
| 8.3 【例 2】WHILE ループを使ったシェルスクリプト.....                        | 47        |
| 8.4 【例 3】第 6 章の宿題 2 を一つの BOURNE シェルスクリプトで処理する.....         | 48        |
| 宿題.....                                                    | 50        |
| <b>APPENDIX A XMING のインストールと使い方.....</b>                   | <b>51</b> |
| A.1 XMING のインストール.....                                     | 51        |
| A.2 XMING の使い方.....                                        | 53        |
| A.3 困った時に.....                                             | 56        |
| <b>APPENDIX B UNIX コンピュータの設定 (管理者権限が不要な事項).....</b>        | <b>57</b> |
| B.1 テキスト教材の導入.....                                         | 57        |
| B.2 INTEL FORTRAN COMPILER を使う場合.....                      | 58        |
| B.3 LITTLE ENDIAN で入出力する場合.....                            | 58        |
| <b>APPENDIX C UNIX コンピュータの設定 (管理者権限が必要な事項).....</b>        | <b>60</b> |
| C.1 テキスト実施にあたって必要な UNIX 環境.....                            | 60        |
| C.2 UNIX 環境構築の事例 (MAC OS 10.6 の場合).....                    | 60        |
| 謝辞.....                                                    | 62        |

## 第 1 章

### UNIX の基本操作(1) 基本的な UNIX コマンド

どうすれば UNIX にログインできるのでしょうか？どうやって UNIX を操作するのでしょうか？第 1 章では UNIX の基本操作について説明します。

#### 1.1 UNIX 環境

このテキストでは次のようなコンピュータ環境を前提にします。まず、UNIX あるいは Linux を含む UNIX 系のオペレーティングシステム (OS) が搭載されたコンピュータ (このテキストでは UNIX コンピュータと呼びます) がネットワークに接続されており、あなたのアカウントが作成されています。あなたは Windows OS が搭載されたコンピュータ (Windows コンピュータと呼びます) を持っており、ネットワークに接続しています。これからあなたは Windows コンピュータから UNIX コンピュータにログインし、UNIX 環境を利用します。あなたの持っている Windows コンピュータをクライアント、UNIX コンピュータをサーバといいます。Windows コンピュータから UNIX コンピュータにログインして UNIX 環境を利用するには PC X サーバと総称されるソフトウェアが必要です。このテキストでは Xming というフリーソフトの利用を前提にします。

Xming のインストールと使い方については Appendix A を、UNIX コンピュータの設定については Appendix B と Appendix C を読んでください。

#### 1.2 ログインとログアウト

##### 【ログイン】

1. Xming を使ってサーバに接続してください (Appendix A を読みましょう)。
2. ユーザ名とパスワードを入力します。
3. 画面にウィンドウが現れれば成功です。このウィンドウを端末と呼びます。

##### 【ログアウト】

端末に Exit と打ち込むか、ウィンドウの「X」ボタンをクリックします。

#### 1.3 UNIX コマンド①——ファイルの管理

Windows ではマウスを使ってアイコンをクリックしてコンピュータを操作します。このような概念を GUI (Graphic User Interface) と言います。これに対して、UNIX ではキーボードを使って端末にコマンドを打ち込んでコンピュータを操作するのが基本です。このような概念を CUI (Command User Interface) と言います。UNIX 操作の基本となるコマンドを表 1-1 から表 1-4 にまとめました。一つずつ端末に打ち込んで機能を確認してみてください。

表 1-1 はファイル管理のためのコマンドです。Windows の Explorer での操作に相当しま

す。以下にディレクトリという言葉があらわれますが、Windows のフォルダと同じ意味です。表の中にある%はプロンプト（ターミナルに表示される「pc090-041[35]」などの文字列のこと）なので、入力しないで下さい。

表 1-1 ファイルの管理(**directory** は任意のディレクトリを、**file** は任意のファイルを示す)

|                                                                                                                                     |                                                                                                                                                                                                             |
|-------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 今いるディレクトリ名を表示する(pwd は present working directory の略)                                                                                 | % pwd                                                                                                                                                                                                       |
| 今いるディレクトリにあるファイル名・ディレクトリ名を表示する(ls は list の略)                                                                                        | % ls                                                                                                                                                                                                        |
| より詳細なファイル、ディレクトリ情報を表示する(l は long の略) <sup>4</sup> 表示される内容は左から順に、アクセス権限, リンク数, 所有者, グループ所有者, ファイルサイズ, 月, 日, 年または時刻, ファイル名 or ディレクトリ名 | % ls -l                                                                                                                                                                                                     |
| ディレクトリを移動する(cd は change directory の略) <sup>5</sup>                                                                                  | % cd <b>directory</b>                                                                                                                                                                                       |
| ひとつ上のディレクトリに移動する(ピリオド2つは一つ上のディレクトリを示す)                                                                                              | % cd ..                                                                                                                                                                                                     |
| ホームディレクトリ <sup>6</sup> に移動する(チルダはホームディレクトリを示す)                                                                                      | % cd ~                                                                                                                                                                                                      |
| 今いるディレクトリに移動する(ピリオド1つは今いるディレクトリを示す)                                                                                                 | % cd .                                                                                                                                                                                                      |
| ファイルの内容を表示する                                                                                                                        | % less <b>file</b><br>% more <b>file</b><br>(1行スクロールするには enter を、<br>1画面スクロールするには space を、<br>終了するには q と入力する)<br>% head <b>file</b><br>(ファイルの先頭の 10 行だけ表示する)<br>% tail <b>file</b><br>(ファイルの末尾の 10 行だけ表示する) |
| ファイルを編集する(新しくウィンドウを開く)                                                                                                              | % emacs <b>file</b>                                                                                                                                                                                         |
| ファイルを編集する(ウィンドウを開かない)                                                                                                               | % emacs -nw <b>file</b>                                                                                                                                                                                     |
| ファイルをコピーする(cp は copy の略)                                                                                                            | 5% cp <b>file1 file2</b><br>(コピー元は <b>file1</b> 、コピー先は <b>file2</b> )                                                                                                                                       |
| ファイルを移動する／名前を変更する(mv は move の略)                                                                                                     | % mv <b>file1 file2</b><br>(移動元は <b>file1</b> 、移動先は <b>file2</b> )                                                                                                                                          |
| ファイルを削除する(rm は remove の略)                                                                                                           | % rm <b>file</b>                                                                                                                                                                                            |
| 新しいディレクトリを作成する(mkdir は                                                                                                              | % mkdir <b>directory</b>                                                                                                                                                                                    |



|                                               |                                                                                                                                            |
|-----------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| make directory の略)                            |                                                                                                                                            |
| 空のディレクトリを削除する (rmkdir は remove directory の略)  | % rmkdir <b>directory</b>                                                                                                                  |
| 空でないディレクトリを削除する (-r は recursive の略で、再帰的という意味) | % rm -r <b>directory</b>                                                                                                                   |
| ディレクトリをコピーする                                  | % cp -r <b>directory1 directory2</b>                                                                                                       |
| パーミッションを変更する (chmod は change mode の略) ※1.7 参照 | % chmod <b>abc file</b><br>a:ユーザの読取権限(+4), 書込権限(+2), 実行権限(+1)<br>b:グループの読取権限(+4), 書込権限(+2), 実行権限(+1)<br>c:その他の読取権限(+4), 書込権限(+2), 実行権限(+1) |
| パーミッションを変更する(もう別の書式) ※1.7 参照                  | % chmod <b>abc file</b><br>a:ユーザ(u),グループ(g),その他(o)<br>b:加える(+), 外す(-)<br>c:読取権限(r), 書込権限(w), 実行権限(x)                                       |
| ファイルを圧縮する(gzip は GNU zip の略)                  | % gzip <b>file</b>                                                                                                                         |
| ファイルを解凍する(gunzip は GNU unzip の略)              | % gunzip <b>file</b>                                                                                                                       |
| 終了する                                          | % exit                                                                                                                                     |

## 1.4 UNIX コマンド②——ファイルの編集

Windows のメモ帳のようにテキストファイルを編集するには Emacs というソフトウェアを使います。Emacs はほとんどの UNIX コンピュータにインストールされているはずです。Emacs はキーボードショートカットを使ってファイルを保存したり、文章をコピーしたりします。表 1-2 には emacs の編集に関するコマンドをまとめました。

表 1-2 emacs を使ったファイルの編集。Ctrl-x とは Ctrl キーと x を同時に押すことを示す。また、Ctrl-x + Ctrl-s は Ctrl キーと x を同時に押したあと、Ctrl キーと s を同時に押すことを示す。

|                                                                 |                     |
|-----------------------------------------------------------------|---------------------|
| Emacs の起動                                                       | % emacs <b>file</b> |
| 保存                                                              | Ctrl-x + Ctrl-s     |
| 終了                                                              | Ctrl-x + Ctrl-c     |
| コマンドのキャンセル<br>(コマンドを打っていてうまく動作しなくなった場合、この操作を2~3回繰り返すと元に戻ることが多い) | Ctrl-g              |
| 文字を削除する(delete)                                                 | Ctrl-d              |

|                      |                                      |
|----------------------|--------------------------------------|
| 行を削除する (kill)        | Ctrl-k                               |
| 切り取り <sup>7</sup>    | Ctrl-Space (始点を指定)<br>Ctrl-w (終点を指定) |
| 貼り付け (yank)          | Ctrl-y                               |
| カーソルを右に移動 (forward)  | Ctrl-f                               |
| カーソルを左に移動 (backward) | Ctrl-b                               |
| カーソルを上に移動 (previous) | Ctrl-p                               |
| カーソルを下に移動 (next)     | Ctrl-n                               |
| 画面を下へスクロールする         | Ctrl-v                               |
| 最後の画面へスクロールする        | Esc + >                              |
| 画面を上へスクロールする         | Esc + v                              |
| 最初の画面にスクロールする        | Esc + <                              |
| 検索                   | Ctrl-s                               |
| 置換                   | Esc + %                              |
| 一括置換                 | Esc-x replace string                 |

## 1.5 UNIX コマンド③——その他のコマンド

表 1-3 には画像ファイルの表示や印刷など、よく使うコマンドをまとめました。

表 1-3 よく使うコマンド

|                                 |                                |
|---------------------------------|--------------------------------|
| Firefox ブラウザ <sup>8</sup> の起動   | % firefox                      |
| JPEG や GIF ファイルの表示 <sup>9</sup> | % display <b>file</b>          |
| 画像ファイルの変換 <sup>10</sup>         | % convert <b>file1 file2</b>   |
| 印刷 <sup>11</sup>                | % lpr -P <b>printer file</b>   |
| 印刷ジョブの表示                        | % lpq -P <b>printer</b>        |
| 印刷ジョブの削除                        | % lprm -P <b>printer jobID</b> |
| コンピュータにログインしている<br>ユーザの一覧を表示する  | % who                          |
| 日付を表示する                         | % date                         |
| <b>year</b> 年のカレンダーを表示する        | % cal <b>year</b>              |
| ファイルの行、単語数、文字数を表示する             | % wc <b>file</b>               |

## 1.6 UNIX コマンド④——困ったときは

困ったときには、表 1-4 のコマンドを試してみましょう。

表 1-4 困ったときのコマンド

|                        |                      |
|------------------------|----------------------|
| コマンドのマニュアルを表示する        | % man <b>command</b> |
| プロンプトが返らなくなってコマンドを中断する | Ctrl-c               |

|                           |                            |
|---------------------------|----------------------------|
| コマンドの状態と process ID を表示する | % top                      |
| コマンドを強制終了させる              | % kill <i>processID</i>    |
| コマンドを強制終了させる(強力)          | % kill -9 <i>processID</i> |

## 1.7 パーミッション

パーミッションは UNIX で非常に重要な概念なので必ず覚えましょう。まず、ユーザのファイルやディレクトリへのアクセスは 3 通りあります。

- 読取：ファイルを開いて読む
- 書込：ファイルを新しく作る。または既存のファイルの内容を改変する。
- 実行：ファイルにコマンドが書かれているとみなして実行する。

次に UNIX では全てのファイルとディレクトリは誰か一人のユーザによって所有されています。またユーザが集まってグループを作ることができます。よってファイルやディレクトリとユーザの関係も 3 通りあります。

- User: ファイル・ディレクトリの所有者であるユーザ
- Group: 所有者ではないが所有者と同じグループに所属するユーザ(group)
- Others: 所有者でなく、かつ所有者と同じグループに所属していないユーザ

おそらく、ファイルの実行というのが分かりにくいと思うので、実際に試してみましょう。emacs を使って次のようなテキストファイルを作成しましょう。

```
pwd
ls
who
```

これを“example1.txt”という名前で作成してください。これは Windows で普通のテキストファイルを作るのと同じですね。

次にパーミッションを変更して、実行権限を付け加えましょう。このファイルは自分で読み、編集し、実行するものです。ただし、同じグループのユーザや他人には読まれてもよいけれども、編集されたり、実行されたりしたくないとします。その時は、

```
% chmod 744 example1.txt
```

とします<sup>12</sup>。あるいは

```
% chmod u+x example1.txt
```

としても同じです<sup>13</sup>。こうすると、あなたはこのファイルが実行可能になります。それでは、example1.txt を実行しましょう。

```
% example1.txt
```

UNIX では Windows と違って、実行権限さえあれば、テキストファイルも実行できることが分かりましたね。この場合、テキストファイルに書かれた内容が、コマンドとして解釈されます。

## 宿題

---

1. 現在のディレクトリを確認しましょう。
2. あなたのホームディレクトリに移動しましょう。(～)
3. “CMAP”という名前のディレクトリを作成しましょう。
4. 作成したディレクトリ“CMAP”に移動しましょう。
5. ~/unix\_semi/lesson1 から
  - global.sh
  - grad.cpt
  - data.xyz をコピーしましょう。
6. global.sh のパーミッションを変更して、実行権限を付け加えましょう。
7. global.sh を実行しましょう。
8. 出力された画像ファイル”image.eps”を表示しましょう。これは次章以降で説明しますが、1979 年 1 月の世界の降水量のデータを示しています。
9. 画像ファイル”image.eps”を EPS 形式から GIF 形式に変換しましょう。

※宿題の解答は~/unix\_semi/lesson1/exercise1.txt にあります。

第 2 章

UNIX の基本操作(2) Redirect/Pipe/Wildcard/Awk

第1章で使ったファイル `global.sh` を開いてください。このファイルは多数の短いコマンドから成り立っています。注意深く見ると、`>`、`>>`、`<`、`<<`、`|` のような特殊な文字がたくさん使われていることがわかります。これは UNIX を操作する上で大変重要な記号です。第2章では、これらの記号について勉強しましょう。

2.1 Redirect

「`<`」と「`>`」の記号はリダイレクト(redirect)と呼ばれます。リダイレクトはコマンドの出力をファイルに書き込みたい場合や、コマンドへの入力をファイルから読み込みたい場合に用います。リダイレクトの種類を表 2-1 にまとめました。

表 2-1 リダイレクトの種類

|                                                           |                                                          |
|-----------------------------------------------------------|----------------------------------------------------------|
| コマンドの出力をファイルに書き込む                                         | % <i>command</i> > <i>file</i>                           |
| コマンドの出力をファイルの末尾に書き足す                                      | % <i>command</i> >> <i>file</i>                          |
| コマンドの出力をファイルに書き込む。<br>もとのファイルは上書きされてなくなる <sup>14</sup>    | % <i>command</i> > <i>file</i>                           |
| ファイルを入力としてコマンドを実行する                                       | % <i>command</i> < <i>file</i>                           |
| 次に「EOF」という文字列が入力されるまで<br>キーボード入力された文字列を<br>入力としてコマンドを実行する | % <i>command</i> << EOF<br><br><i>strings</i><br><br>EOF |

【例 1】

ターミナルに次のように打ち込んでみましょう。`date` は現在の日付や時刻を出力するコマンド、`wc` は文字数や行数を数えるコマンドで、行の数、単語の数、文字の数の 3 つが出力されます。

```
% date > redirect.txt
% more redirect.txt
% date >> redirect.txt
% more redirect.txt
% date > redirect.txt
% more redirect.txt
% wc < redirect.txt
```

エラーが出るときは  
% date >! redirect.txt  
を試みましょう

```
% wc << EOF
Hello.
My name is Naota.
EOF
```

```
% wc << EOF >> redirect.txt
Hello.
My name is Naota.
EOF
% more redirect.txt
```

## 【宿題1】

1. 今いるサーバにログインしている人の一覧を書き込んだファイルを作りましょう。  
(表 1-3 にあるコマンド 「who」 をうまく使いましょう)
2. 2004 年から 2005 年までのカレンダーを書き込んだファイルを作りましょう。(表 1-3 にあるコマンド 「cal」 をうまく使いましょう)

※宿題1の解答は~/unix\_semi/lesson2/exercise1.sh にあります。

## 2.2 Pipe

「|」の記号はパイプ(pipe)と呼ばれます。パイプはコマンドの出力を次のコマンドへの入力として受け渡します(表 2-2)。

表 2-2 Pipe

|                                                  |                                     |
|--------------------------------------------------|-------------------------------------|
| まずコマンド1が実行され、<br>次にコマンド1の出力を入力として<br>コマンド2が実行される | % <i>command1</i>   <i>command2</i> |
|--------------------------------------------------|-------------------------------------|

## 【例2】

```
% cd ~/unix_semi/dat_bin
% ls -l
% ls -l | more
% ls -l | tail
% ls -l | head
```

## 【宿題2】

1. あなたのホームディレクトリにあるファイルとディレクトリを数を数えましょう。  
(コマンド「wc」をうまく使いましょう)

- `redirect` を使ってやってみましょう
- `pipe` を使ってやってみましょう

※宿題 2 の解答は `~/unix_semi/lesson2/exercise2.sh` にあります。

## 2.3 Wildcard

「?」と「\*」の記号はワイルドカード(Wildcard)と呼ばれ、それぞれ任意の 1 文字や文字列を表します (表 2-3)。

表 2-3 wildcard

|                              |   |
|------------------------------|---|
| Null <sup>15</sup> を含む任意の文字列 | * |
| 任意の 1 文字                     | ? |

### 【例3】

```
% cp -r ~/unix_semi/lesson2/wildcard .
% cd wildcard
% ls -l
% ls -l *.txt
% ls -l test*
% ls -l te?t1.txt
```

ピリオドを  
忘れないで

t の後は数字

### 【宿題 3】

1. 例 3 でコピーしたディレクトリ「`wildcard`」に移動しましょう。
2. ファイル名に「1 (数字)」を含むファイルを削除しましょう。
3. ディレクトリ `wildcard` の中のすべてのファイルを削除しましょう
4. 例 3 のように「`-r`」オプションを使わずに、`~/unix_semi/lesson2/wildcard` からすべてのファイルをコピーしましょう。(「\*」をうまく使きましょう)

※宿題 3 の解答は `~/unix_semi/lesson2/exercise3.sh` にあります。

## 2.4 AWK

AWK は一種のプログラミング言語で、主にテキストを処理するために使われます。コンパイルする必要がないインタプリタ言語で、直接ターミナルに打ち込んで使えます。文法は C 言語とよく似ています。AWK を使えばかなり高度なプログラミングも可能ですが、ここでは表 2-4 の 2 つの使い方だけ覚えましょう。表 2-4 で「`'`」と「`'`」で囲まれた部分が AWK の解釈するソースコードです。

表 2-4 AWK のコマンド

|                                        |                                                                          |
|----------------------------------------|--------------------------------------------------------------------------|
| ファイルの $N_1, N_2$ 列目のデータを書き出す           | % <code>awk '{print \$<math>N_1</math>, \$<math>N_2</math>}' file</code> |
| M 列目が m の時、N 列目のデータを書き出す <sup>16</sup> | % <code>awk '(\$<math>M</math>==m){print \$<math>N</math>}' file</code>  |
| 例) 1 列目が "a" のとき、2 列目のデータを書き出す。        | % <code>awk '(\$1=="a"){print \$2}' file</code>                          |
| 例) 1 列目が 12 のとき、2 列目のデータを書き出す。         | % <code>awk '(\$1==12){print \$2}' file</code>                           |

## 【例 4】

```
% cd ~
% ls -l > home.txt
% awk '{print $9}' home.txt
% awk '($5==512){print $9}' home.txt
```

さて第 1 章で使ったファイル「data.xyz」をもう一度開いてみてください。これは CMAP の 1979 年 1 月の降水量データを 1 列目に経度、2 列目に緯度、3 列目に降水量（単位は mm/day）の順で記録したデータです。CMAP は 2.5 度×2.5 度で全球をカバーするので、全部で 144×72=10368 行あります。次の第 3 章で解説しますが、global.sh は data.xyz を読み込んで描画を行います。つまり、第 1 章で作成した図は data.xyz を可視化したものだったのです。

次に CMAP のウェブサイト (<ftp://ftp.cpc.ncep.noaa.gov/precip/cmap/monthly/>) で配布されているファイルを見てみましょう。まず、1979 年の月単位のデータファイルをダウンロードしましょう<sup>17</sup>（※Appendix B 参照）。このファイルは圧縮されているので解凍しましょう。gz 圧縮ファイルの解凍には gunzip コマンドを使います。

```
% gunzip cmap_mon_v0705_79.txt.gz
```

さて、ファイルの内容を表示させましょう。図 2-1 に凡例を示しますが、data.xyz とは書式がかなり異なることが分かるでしょう。

```
% more cmap_mon_v0705_79.txt
```



| year | month | lat    | lon   | data | See the CMAP documents |      |       |
|------|-------|--------|-------|------|------------------------|------|-------|
| 1979 | 1     | -88.75 | 1.25  | 0.08 | 60.00                  | 0.08 | 60.00 |
| 1979 | 1     | -88.75 | 3.75  | 0.07 | 60.00                  | 0.07 | 60.00 |
| 1979 | 1     | -88.75 | 6.25  | 0.07 | 60.00                  | 0.07 | 60.00 |
| 1979 | 1     | -88.75 | 8.75  | 0.07 | 60.00                  | 0.07 | 60.00 |
| 1979 | 1     | -88.75 | 11.25 | 0.07 | 60.00                  | 0.07 | 60.00 |
| 1979 | 1     | -88.75 | 13.75 | 0.07 | 60.00                  | 0.07 | 60.00 |

図 2-1 CMAP ファイルの凡例

## 【宿題 4】

- 図 2-1 にあるのがあなたのダウンロードした `cmap_mon_v0705_79.txt` の凡例です。これを参考にして、`AWK` を使って `data.xyz` と全く同じ形式のファイルを作りましょう。ここで注意することは、`data.xyz` は 1979 年 1 月のデータを経度、緯度、降水量という書式で書き出したファイルです。

※ 宿題 4 の解答は `~/unix_semi/lesson2/exercise4.sh` にあります。

※ 出力の桁数を揃えたい場合は、インターネット検索してください。

## 第3章

### Bourne シェルスクリプト(1) GMT コマンドを組み合わせる

第1章では、`global.sh` というファイルを実行して地図を描きましたが、この地図はどのようにして描かれたのでしょうか。どのようにすれば、編集できるのでしょうか。

#### 3.1 GMT とは

第1章の復習をしましょう。以下のようなコマンドを打ち込み、図 3-1 のような地図を出力したのです。

```
% global.sh
% display image.eps
```

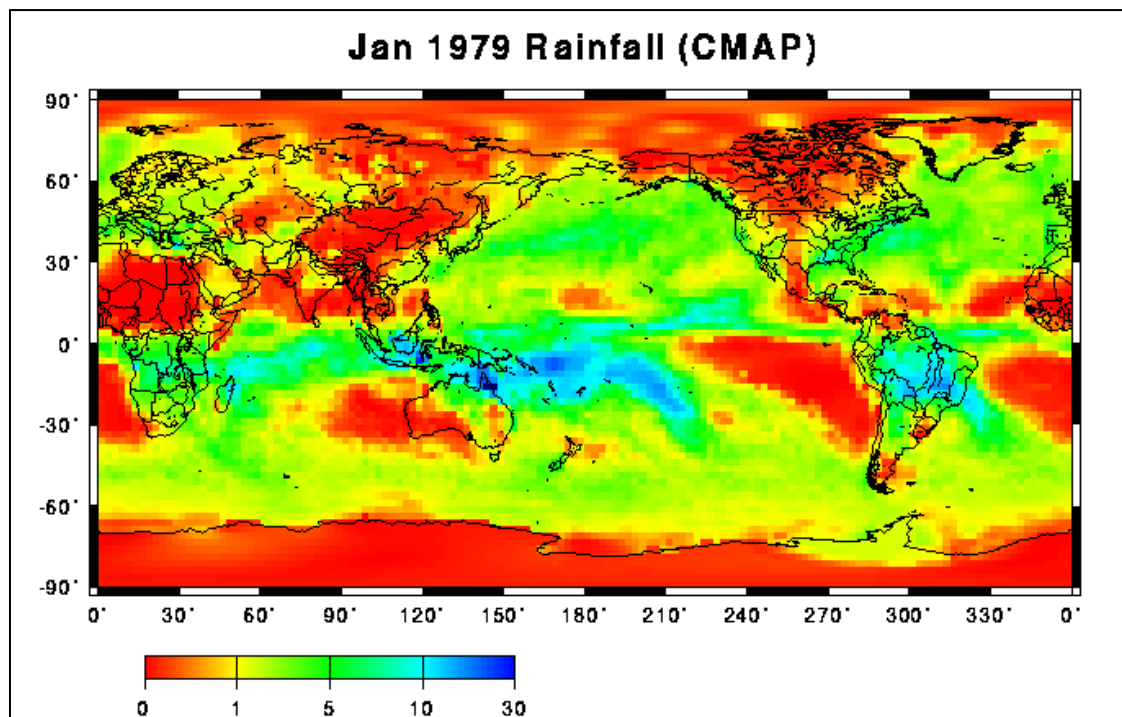


図 3-1 `global.sh` で作成された図

この作図には Generic Mapping Tool (GMT)<sup>18</sup>というフリーソフトウェアが使われています。GMT は `ls` や `cd` と同じような形式の多数のコマンドからなります。`global.sh` ではこれらの GMT コマンドを組み合わせることにより、全球の降水量の地図を描いていたのです。

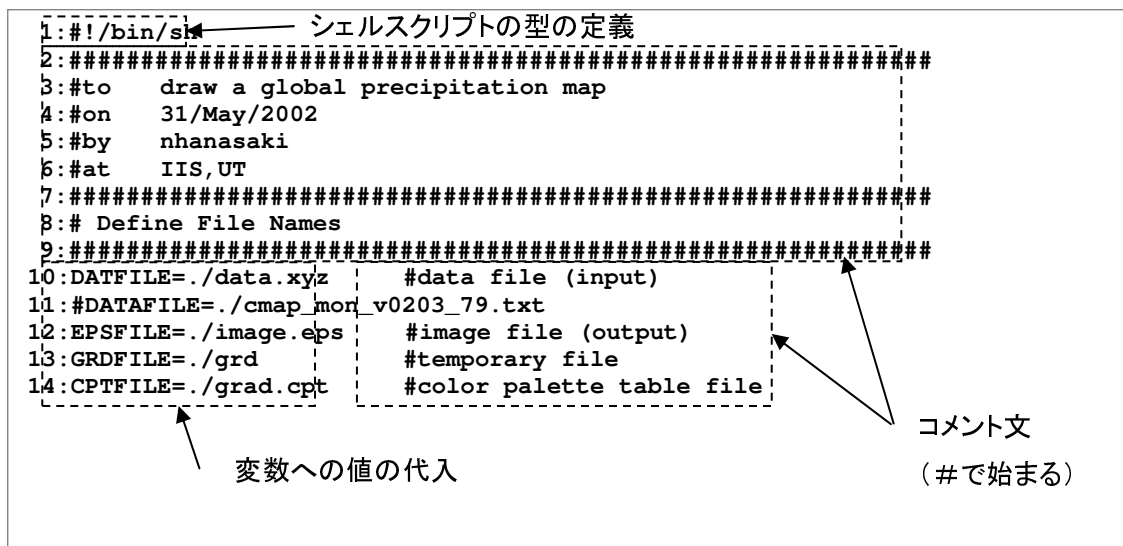
GMT を利用して作図するには以下の 3 つのファイルが必要です。

- ソースファイル (`global.sh`)
- データファイル (`data.xyz`)
- CPT ファイル (`grad.cpt`)

それぞれのファイルの役割を解説していきます。

## 3.2 GMT のシェルスクリプト

global.sh をテキストエディタで開いてみましょう。1.7 で pwd, ls, who とコマンドを書いたテキストファイルに実行権限を加えて実行したことを覚えているでしょう。この global.sh も同じで、たくさんのコマンドが書きこまれています。ただし、global.sh は Bourne Shell という文法に従って記述されており、それによってコマンドの実行が制御できるようになっています。こうしたファイルを Bourne シェルスクリプトと言います。つまり、global.sh は GMT コマンドを含む Bourne シェルスクリプトである、ことができます。以降では、ファイルを3つのブロックに区切って、解説していきます。それぞれ第1ブロック、第2ブロック、第3ブロックと呼ぶことにしましょう。



第1ブロックについて説明しましょう。1行目に「#!/bin/sh」と書いてありますが、これはこのファイルが Bourne シェルスクリプトであることを示す文字列です。2行目から9行目まで「#」で始まる行が続きますが、「#」以降の文字列はコンピュータに解釈されません。よって、この例のように、覚え書きや区切りをいれるために使われます。こうした文をコメント文と言います。

10行目から14行目では、2つの文字列が「=」で左右につながっていますね。「=」の左側の文字列は「変数」です。空の箱だと考えてください。「=」の右側の文字列は「値」です。何かの物だと考えてください。「変数」と「値」を「=」で結ぶことにより、「変数」に「値」を代入する、つまり「変数」という空の箱に、「値」という物を入れることができます。たとえば10行目に「DATFILE=./data.xyz」とあるのは、「DATFILE」という変数(空き箱)に「./data.xyz」という値(物)を入れたことになります。ここではファイルの名前が定義されています。<sup>19</sup>

```

1:#####
2:# Define Mapping Area
3:#####
4:XMIN=0.00          #Horizontal minimum [degree]
5:XMAX=360.00        #Horizontal maximum [degree]
6:YMIN=-90.00        #Vertical minimum [degree]
7:YMAX=90.00         #Vertical maximum [degree]
8:XWID=21.0          #Width of image [cm]
9:YWID=10.5          #Height of image [cm]
10:DXa=30.0           #a:Horizontal Annotation Interval [degree]
11:DXf=30.0           #f:Horizontal Frame Interval [degree]
12:DXg=10.0           #g:Horizontal Grid Interval [degree]
13:DYa=30.0           #a:Vertical Annotation Interval [degree]
14:DYf=30.0           #f:Vertical Frame Interval [degree]
15:DYg=10.0           #g:Vertical Grid Interval [degree]
16:D=2.5              #grid size
17:#####
18:# GMT Options
19:#####
20:RFLAG=-R${XMIN}/${XMAX}/${YMIN}/${YMAX}
21:JFLAG=-JX${XWID}d/${YWID}d
22:BFLAG=-Ba${DXa}f${DXf}g${DXg}:Longitude:/a${DYa}f${DYf}g${DYg}:Latitude:news

```

変数 DYa の値で置き換え

第2ブロックに移りましょう。1行目から16行目にかけては、引き続き変数に値が代入されています。ここで定義されているのは描画範囲です。それぞれの変数と値の意味は、図3-2を参考にしてください。

17行目から22行目で定義されているのは GMT で利用する -R, -J, -B オプションです。ここで「=」の右側に「\${ }」という記号がありますね。これは変数に代入されている値で置き換えることを示します。例えば\${XMIN}と書いてあったら、それを変数 XMIN に代入されている値で置き換えることを意味します。4行目で変数 XMIN には 0.00 という値が定義されていますから、-R\${XMIN}/は-R0.00/に置き換えられるというわけです。

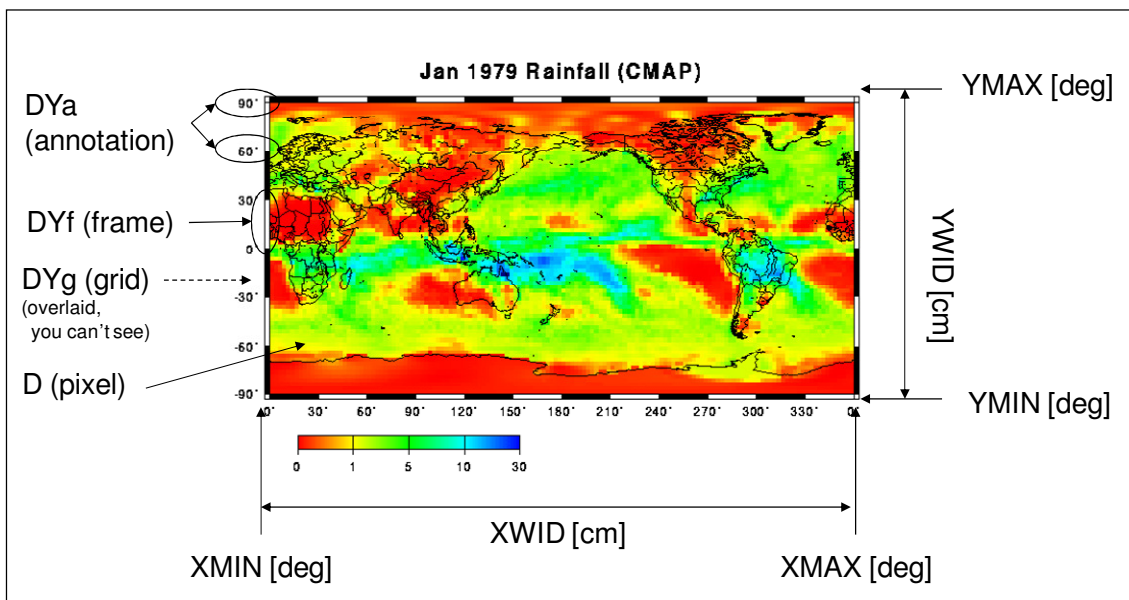


図 3-2 GMT の描画範囲

```

1:#####
2:###---Jobs
3:#####
4:awk '($2==1){print $4, $3, $5}' $DATAFILE | \
5:xyz2grd $RFLAG -G$GRDFILE -I${D}/${D} -F
6:xyz2grd $RFLAG -G$GRDFILE -I${D}/${D} -F $DATAFILE
7:##
8:psbasemap $RFLAG $JFLAG $BFLAG -X5.0 -Y5.0
9:grdimage -O $RFLAG $JFLAG $GRDFILE -C$CPTFILE
10:pscoast -O $RFLAG $JFLAG -Dc -W5 -N1 -I1
11:psscale -O -C$CPTFILE -D5.0/-1.5/8/0.5h -L
12:pstext -O $RFLAG $JFLAG -N
13:180 110 24 0 1 6 Jan 1979 Rainfall (CMAP)
14:EOF

```

リダイレクト (第2章参照)

EOF まで読み込む

trailer を省略

ヘッダーを省略

コマンド

第3ブロックにはコマンドが書かれています。ここがこのシェルスクリプトの本体と言えます。20 6行目が最初のコマンドです。`$RFLAG` とあるのは<sup>21</sup>、変数 `RFLAG` に代入された値で置き換えるということなので、この行は

「xyz2grd -R0.00/360.00/-90.00/90.00 -G/grd -I2.5/2.5 -F」

と書いた場合と全く同じになります。ここで、`xyz2grd` がコマンドで、「-」(ハイフン) から始まる文字列がオプションです。第1章で「ls -l」というコマンドを勉強しましたが、ls というファイルを表示するコマンドに、-l というオプションを加えることで、詳細なリストを表示するように機能が拡張することができました。8行目の `psbasemap` 以降のコマンドではリダイレクト(>)がありますね。忘れてしまった使い方は、第2章を復習しましょう。

さて、第3ブロックではどのような作業が行われているのでしょうか。概念図で表すと図3-3のようになります。

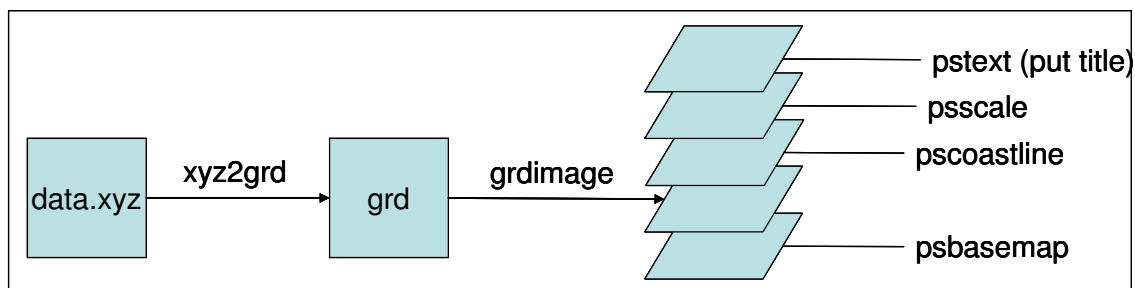


図 3-3 global.sh で行われる作業

まず、`xyz2grd` というのは「経度、緯度、データ」という書式で書かれた1地点ごとのテキストファイル(変数は `DATFILE`、値は `data.xyz`)を地図のような2次元情報を持つグリッドデータに変換する `GMT` コマンドです。この結果、中間ファイル(変数は `GRDFILE`、値は `./grd`)が作成されます。`psbasemap`、`grdimage`、`pscoast`、`psscale`、`pstext` が作図コマンドです。`GMT` は `OHP` シートやトレーシングペーパーを重ねるようにして作図していきます。`psbasemap` はその名の通り、図の下絵を描くコマンドです。作成した下絵は出力画像ファイル(変数は `EPSFILE`、値は `image.eps`)に書き込まれます。`grdimage` は `xyz2grd` で作った中間ファイルを描画するコマンドです。降水量に応じた色はここで設定され、出力画像ファイルに書き込まれます。`pscoast` は海岸線や国境線、河川を、`psscale`

はカラーバーを、`pstext` は文字列（タイトルなど）を描くコマンドです。 `x`, `y`, `size`, `angle`, `fontno`, `justify` の順で引数を与えます。`size` はフォントの大きさ、`angle` は水平から反時計回りに計った角度、`fontno` はフォント番号（種類）、`justify` は文字列の配置をそれぞれ意味しています。これらの GMT コマンドが順々に出力画像ファイルに書き込まれていきます。

### 3.3 GMT コマンドのオプション

GMT コマンドでよく使うオプションを表 3-1 にまとめました。これらの詳しい使い方は、GMT のマニュアルを参照してください。すぐれた web サイトもいくつも公開されているので、google 等で検索してみてもよいでしょう。

ここで特に重要なのが「-O」と「-K」の2つのオプションです。GMT では OHP シートを重ねるようにして図を描きます。そのため、どの OHP シートが一番下で、どれが一番上かを示すことがとても重要です。GMT コマンドによる出力が、最初の OHP シートでない場合、オプション「-O」をつけなければなりません。「-O」は Overlay plot mode の略で、1 枚目の OHP であることを示すヘッダー情報を省略することを示します。同様に、最後の1枚でない場合「-K」をつけなければなりません。「-K」は More PostScript code will be appended later の略で、最後の1枚であることを示すフッター情報を省略することを示します。

表 3-1 GMT コマンドのオプション

| 複数の GMT コマンドに共通するオプション |                             |
|------------------------|-----------------------------|
| -R                     | 描画領域を指定する                   |
| -J                     | 投影法を指定する                    |
| -B                     | 軸書式を指定する                    |
| -O                     | 最初の1枚目であることを示す header を省略する |
| -K                     | 最後の1枚であることを示す trailer を省略する |
| psbasemap のオプション       |                             |
| -X                     | x 方向に原点を移動する(単位:cm)         |
| -Y                     | y 方向に原点を移動する(単位:cm)         |
| pscoast のオプション         |                             |
| -D                     | 地図の解像度                      |
| -W                     | 線の太さ                        |
| -N                     | 国境線の描画                      |
| -I                     | 河川の描画                       |
| psscale のオプション         |                             |
| -D                     | カラーバーの位置と大きさを指定する           |
| -L                     | カラーバーの間隔を等間隔にする             |

| pstext のオプション |                         |
|---------------|-------------------------|
| -N            | basemap の外側でも文字列を表示します。 |

### 3.4 CPT ファイル

図 3-1 では降水量が小さいところは赤、大きいところは青で示されています。それぞれの色と降水量との対応は左下のカラーバーに示されていますね。色が設定されているのは降水量が 0 から 30 までの間で、赤、黄色、緑、シアン、青の 5 色が割り当てられていることが分かります。この色の設定をしているのが CPT (Color Palette File) ファイルです。図 3-1 で使われた CPT ファイルは「grad.cpt」です。

第 1 章で使った grad.cpt を開いてみましょう。図 3-4 のようになっているはずです。CPT ファイルは基本的に 1 行が 8 列の数値からなります。それぞれ、開始値、R、G、B、終了値、R、G、B の順です。例えば開始値 0 のとき赤 (RGB で 255/0/0) を、終了値 1 のとき黄 (RGB で 255/255/0) で表示し、その間を段階的に色を変えたい場合、

0      255      0      0      1      255      255      0

と書きます。このとき値の区切りには必ず tab を使ってください。space を使うとエラーになりますので注意して下さい。値が最小値 (0) より小さい値、最大値 (30) より大きい値、実数以外の値に色を割り当てるときはそれぞれ B (Background)、F (Foreground)、N (Not a number) の後に RGB を入力します。「grad.cpt」の場合、0 より小さいときは濃い赤、30 より大きい時は濃い青の色が設定されています。

| Start value | End value | R   | G   | B   | R   | G   | B   |
|-------------|-----------|-----|-----|-----|-----|-----|-----|
| 0           | 1         | 255 | 0   | 0   | 255 | 255 | 0   |
| 1           | 5         | 255 | 255 | 0   | 0   | 255 | 0   |
| 5           | 10        | 0   | 255 | 0   | 0   | 255 | 255 |
| 10          | 30        | 0   | 255 | 255 | 0   | 0   | 255 |
| F           |           | 0   | 0   | 128 |     |     |     |
| B           |           | 128 | 0   | 0   |     |     |     |
| N           |           | 255 | 255 | 255 |     |     |     |

<TAB> spacing  
Do not use <SPACE> spacing !!

Color (RGB)

0 1 5 10 30

図 3-4 grad.cpt

### 宿題

global.sh のスクリプトを変更して、以下のような図を描いてみましょう。

1. 図のタイトルを変えましょう。(どのように変えても結構です。)
2. 図のカラーバーには単位が書いていませんね。カラーバーの適当なところに、単位 [mm/day] を入れましょう。
3. 図を拡大してイランを拡大表示しましょう。水平方向は 60 度、鉛直方向は 30 度としてください。次に、イランは雨が少ないので、現在の CPT ファイル (grad.cpt) では、国内の雨の分布がはっきりしません。そこで、CPT ファイルを編集して、国内分布がはっきりするような図にしてください。最後に図のタイトルを変え、単位 [mm/day] も入れましょう。

※宿題の答えは、~/unix\_semi/lesson3 にあります。



## 第 4 章

### グリッドデータの処理(1) FORTRAN プログラムを利用した演算

この章では CMAP データを用いて全球の降水量の解析をしてみましょう。CMAP のデータは全球、 $2.5^{\circ} \times 2.5^{\circ}$  解像度のグリッドデータです。また、1979 年から 2002 年まで、月単位のデータが配布されています。この章では、1987 年と 1988 年の年間降水量を比較した図を作図してみましょう。ちなみに、1987 年はエルニーニョだった年、1988 年はラニーニャだった年として知られています。

#### 4.1 アスキーとバイナリ

第 2 章で見た通り、CMAP データはテキストデータとして配布されています。配布されていたデータを全てダウンロードし、バイナリ (binary) という形式に変換したものが `~unix_semi/dat_bin` にあります。この章ではこのバイナリデータを使います。

計算機で情報を保存する方法は、基本的に 2 つしかありません。アスキーかバイナリのどちらかです。アスキーはテキストとも呼ばれます。アスキーデータは Windows のメモ帳などで読むことができます。これと反して、バイナリは機械語なのでメモ帳では読めません。それぞれの特徴を表 4-1 にまとめました。

表 4-1 アスキーとバイナリの比較

|                      | アスキー      | バイナリ      |
|----------------------|-----------|-----------|
| メモ帳で読めるか？            | Yes       | No        |
| マシン依存性 <sup>22</sup> | No        | Yes       |
| データサイズ               | 1 byte/文字 | 4byte/データ |

アスキーデータは一つの文字が 1byte で記録されますが<sup>23</sup>、バイナリデータでは、ひとつの実数を 4byte で記録することができます。例えば、実数「1.0」を保存するのに必要な記憶容量はアスキーの場合 3 byte ですが、バイナリでは 4 byte になります。実数「1.000000」を保存するのに必要なバイト数は 8 byte ですが、バイナリではやはり 4 byte となります。

CMAP のデータは  $2.5^{\circ} \times 2.5^{\circ}$  (経度×緯度) の解像度があるので、一月分のデータは  $144 \times 72$  の実数で構成されることになります。このデータを図 4-1 のように横に 144 列、縦に 72 行並べて保存したとすると、ファイル容量は  $144 \times 72 \times 4 = 41472$  バイトになります。実数であれば、値が 1.0 でも、1.0000 でも、 $9.87\text{E}+9$  でも 4byte で表わされます。このため、`~unix_semi/dat_bin` にあるどの年のどの月のデータも全て 41472 byte であることが分かるでしょう。ファイル名は `CMAP203_19870100.cmap` のように表されていますが、これは、(CMAP の ver2.03 の、1987 年、1 月の月単位) のデータであることを表しています<sup>24</sup>。

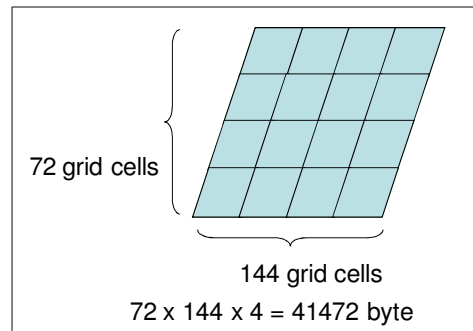


図 4-1 CMAP のバイナリデータ

## 4.2 月単位データの年単位データへの変換

さあ、本題に移りましょう。この章の目的は 1987 年と 1988 年の年降水量を比較することでした。今は月単位のデータしかありませんから、CMAP の 1987 年と 1988 年の月単位のデータを年単位のデータに変換し、その差をとることにしましょう。

まず年単位の降水量データを作成しましょう。やり方は図 4-2 に示すように 1 月から 12 月までのデータを足し、最後に 12 で割ればよさそうですね。

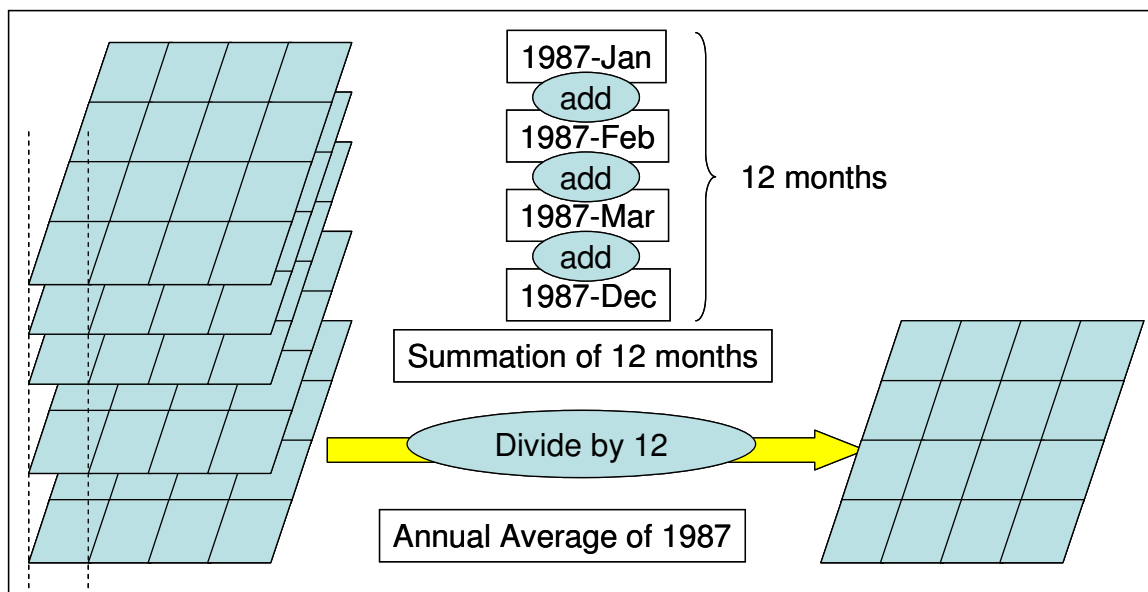


図 4-2 月単位から年単位へのデータ変換

このような作業を行うためにはバイナリファイル同士を足したり、割ったりする特別なコマンドが必要です。表 4-2 にあるようなコマンド<sup>25</sup>が使えるようになっています。これらのコマンドを使うときには、どのファイルを読んで、どのファイルに書き出すかなどを指定する必要があります。例えば `temp.cmap` という 144 列 72 行が実数 0 で満たされたバイナリファイルを新しく一つ作るときは、`createcmap` というコマンドを次のように使います。

```
% createcmap temp.cmap 0
```

このようにすると `createcmap` というコマンドに、`temp.cmap` と 0 という情報が渡されま

す。この temp.cmap や 0 のことを引数(argument)と言います。表 4-2 のコマンドに関しては、引数の順序を変えてはいけません。

表 4-2 バイナリファイル进行操作するためのコマンド(その1)

| プログラム                                                                                                                                                                                                                                                                                                                                                                                             | 引数                                      | 機能                                                 |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------|----------------------------------------------------|
| createcmap                                                                                                                                                                                                                                                                                                                                                                                        | OUT,num                                 | num という実数で満たされたバイナリ OUT を作成する                      |
| addcmap                                                                                                                                                                                                                                                                                                                                                                                           | IN1,IN2,OUT                             | 2つのバイナリ (IN1,N2)の和を求め、バイナリ OUT に書き出す               |
| subcmap                                                                                                                                                                                                                                                                                                                                                                                           | IN1,IN2,OUT                             | 2つのバイナリ (IN1,N2)の差を求め、バイナリ OUT に書き出す               |
| mulcmap                                                                                                                                                                                                                                                                                                                                                                                           | IN,num,OUT                              | バイナリ(IN)に実数値(num)をかけ、バイナリ OUT に書き出す                |
| divcmap                                                                                                                                                                                                                                                                                                                                                                                           | IN,num,OUT                              | バイナリ (IN)を実数値(num)で割り、バイナリ OUT に書き出す               |
| proccmap                                                                                                                                                                                                                                                                                                                                                                                          | IN1,IN2,OUT                             | 2つのバイナリ (IN1,N2)の積を求め、バイナリ OUT に書き出す               |
| ratcmap                                                                                                                                                                                                                                                                                                                                                                                           | IN1,IN2,OUT                             | 2つのバイナリ (IN1,N2)の商を求め、バイナリ OUT に書き出す               |
| asc2cmap                                                                                                                                                                                                                                                                                                                                                                                          | IN,OUT                                  | テキスト(IN)をバイナリ(OUT)に変換する                            |
| cmap2asc                                                                                                                                                                                                                                                                                                                                                                                          | IN,OUT                                  | バイナリ(IN)をテキスト(OUT)に変換する                            |
| xyz2cmap                                                                                                                                                                                                                                                                                                                                                                                          | IN,OUT                                  | lon lat data 形式のテキスト(IN)をバイナリ(OUT)に変換する            |
| cmap2xyz                                                                                                                                                                                                                                                                                                                                                                                          | IN,OUT                                  | バイナリ(IN)から(lon lat data)のテキスト(OUT)に変換する            |
| shifcmap                                                                                                                                                                                                                                                                                                                                                                                          | IN,OUT                                  | バイナリ(IN)を 180°東西方向にずらして、OUT に書き出す                  |
| upsideowncmap                                                                                                                                                                                                                                                                                                                                                                                     | IN,OUT                                  | バイナリ(IN)を南北逆転して、OUT に書き出す                          |
| cmap2eps                                                                                                                                                                                                                                                                                                                                                                                          | IN,CPT,OUT,[title1],[title2]            | バイナリ(IN)を EPS 画像 (OUT)に変換する                        |
| eps2gif                                                                                                                                                                                                                                                                                                                                                                                           | IN,OUT [rot]                            | EPS 画像(IN)を他の画像フォーマット(OUT)に変換する。rot を付けると 90°回転する。 |
| mon2yearcmap                                                                                                                                                                                                                                                                                                                                                                                      | DIR, PRJ, RUN,<br>yearmin, yearmax, suf | 月単位のデータを年単位に変換する                                   |
| noyday                                                                                                                                                                                                                                                                                                                                                                                            | year mon                                | year 年 mon 月の日数を返す。                                |
| <p>ここで、</p> <p>IN: 入力するファイル、OUT: 出力されるファイル、CPT: cpt ファイル</p> <p>num: 実数、year, mon: 年と月を表す整数</p> <p>[title1], [title2]:タイトル of 文字列。「[ ]」で囲まれた引数は省略が可能。</p> <p>yearmin, yearmax: 処理開始年と処理終了年。西暦表示。4桁の整数</p> <p>DIR: ファイルのあるディレクトリ(今いるディレクトリにファイルがあるときは./とする。)</p> <p>PRJ: プロジェクト名を表す4文字の文字列(このテキストでは CMAP)</p> <p>RUN: 実験名を表す4文字の文字列(このテキストでは 203_)</p> <p>suf:拡張子(suffix)を表す文字列(このテキストでは.cmap)</p> |                                         |                                                    |

それでは、表 4-2 にあるコマンドを使って作業を進めましょう。まず、ホームディレクトリに BINARY というディレクトリを作り、そこに 1987 年から 1988 年のバイナリデータをコピーしましょう。

```
% mkdir ~/BINARY
% cd ~/BINARY
% cp ~/unix_semi/dat_bin/*1987* .
% cp ~/unix_semi/dat_bin/*1988* .
```

ただ、実際は~/unix\_semi/dat\_bin/に既に 1987 年と 1988 年の年平均ファイルが用意されています。それも一緒にコピーしてしまったので、消しておきましょう。

```
% rm CMAP203_19870000.cmap
% rm CMAP203_19880000.cmap
```

それでは図 4-2 に示されたように、作業を進めましょう。まず 1987 年の年平均値ファイル「CMAP203\_19870000.cmap」を新しく作ります。

```
% createmap CMAP203_19870000.cmap 0
```

この「CMAP203\_19870000.cmap」は 144 列×72 行の実数からなるバイナリファイルで今は、すべてに 0 の値が入っています。このファイルに、1987 年の 1 月のデータを足しましょう。

```
% addcmap CMAP203_19870000.cmap CMAP203_19870100.cmap CMAP203_19870000.cmap
```

次にこのファイルに 2 月のデータも足します。

```
% addcmap CMAP203_19870000.cmap CMAP203_19870200.cmap CMAP203_19870000.cmap
```

これで、1 月と 2 月の和が CMAP203\_19870000.cmap に入ったことになりますね。これを 1 2 月まで続けていきます。

```
% addcmap CMAP203_19870000.cmap CMAP203_19871200.cmap CMAP203_19870000.cmap
```

最後に 1 2 で割りましょう。

```
% divcmap CMAP203_19870000.cmap 12 CMAP203_19870000.cmap
```

これで、1987 年の平均値を得ることができました。同様に 1988 年の平均値を作ってみましょう。

さて、12 カ月分のファイルを足して、最後に 12 で割ることで、年平均を求めてきましたが、このやり方は実は正確ではありません。なぜなら各月の日数が異なるからです。正確に計算するには、

```
% mulcmap CMAP203_19870100.cmap 31 temp.cmap
```

のように降水量に各月の日数をかけたのち、

```
% addcmap CMAP203_19870000.cmap temp.cmap CMAP203_19870000.cmap
```

として足し合わせ、最後に

```
% divcmap CMAP203_19870000.cmap 365 CMAP203_19870000.cmap
```

と 1 年の日数で割ると正しい計算になります。しかし毎月の日数や閏年を考慮するのは大変です。そこで月単位のデータを年単位のデータに正確に変換する `mon2yearcmap` というコマンドがあります。以下のようにすると、1987 年から 1988 年までの年平均値を正確に計算することができます。

```
% mon2yearcmap ./ CMAP 203_ 1987 1988 .cmap
```

### 4.3 データの差と図化

それでは、1987 年と 1988 年の差をとってみましょう。図 4-3 に示されたように 1987 年のデータから 1988 年のデータを引いて、結果を `dif.cmap` というファイルに書き出してみましよう。

```
% subcmap CMAP0203_19870000.cmap CMAP0203_19880000.cmap dif.cmap
```

この結果を地図にしましょう。ここで差を表示するための「`dif.cpt`」という CPT ファイルを自分で作ってみてください。`cmap2eps` コマンドの機能を使って `Difference` という主タイトルと 1987-1988 という副タイトルをつけましよう。

```
% cmap2eps dif.cmap dif.cpt dif.eps Difference 1987-1988
```

図 4-3 にあるような差が見られましたか？

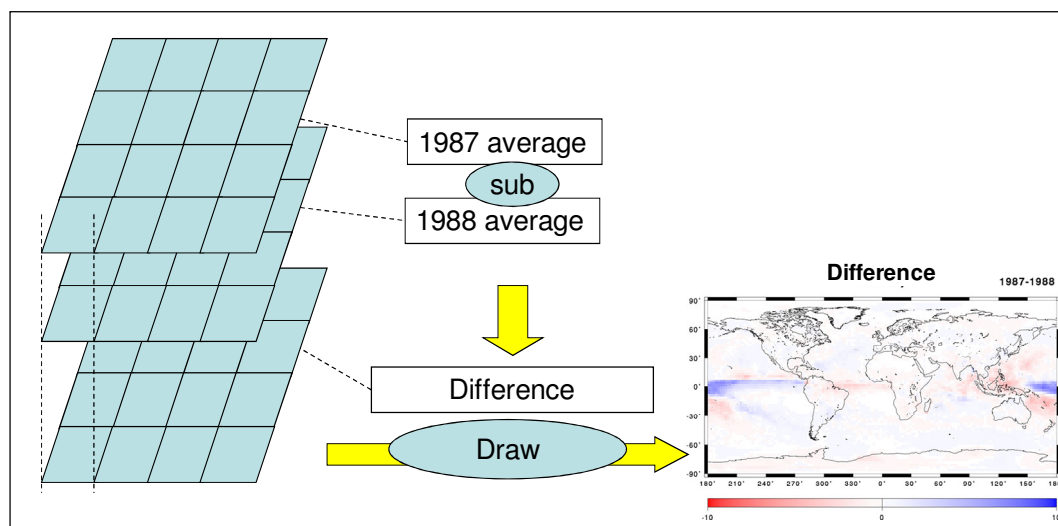


図 4-3 1987 年と 1988 年の差

### 宿題

1. 1987 年と 1988 年の JJA (June- July- August) と DJF (December- January- February)で降水量データを作成し、違いを表す図を作りましよう（自分のオリジナルの CPT ファイルを作りましよう）。ここで 1987 の DJF とは 1986 年 12 月から 1987 年 2 月まで、

1988 年の DJF とは 1987 年 12 月から 1988 年 2 月までを示すものとします。

2. 例題のやり方だとともとの雨が多いところ（熱帯の海洋など）で、差が目立つような図になってしまいます。たとえば、もともと雨の少ないサハラ砂漠では差がないように見え、雨の多い熱帯で差が大きく見えますが、前者は比較的小さい値から小さい値を引いていて、後者は比較的大きい値から大きい値を引いているためにおこります。そこで、1987 年と 1988 年の降水量の差を 1987 年の降水量に対する割合として比較する図を描きましょう。つまり、1987 年の年平均値が 5mm/day で、1988 年が 6mm/day なら、 $(5-6)/5=-0.2$  となるようにしましょう。オリジナルの CPT ファイルをつくり、図示してみましょう。

※宿題の答えは~/unix\_semi/lesson4 にあります。

※ 出力結果がどうしても異常になる場合は、Appendix B を参照し、Endian を変更してみてください。

## 第 5 章

### FORTRAN(1) FORTRAN の入出力

第4章では、表 4-2 のコマンドを利用してバイナリファイルの計算を行いました。これらのコマンドは FORTRAN というプログラム言語を利用して筆者が自作したものです。この章では FORTRAN を使ってどうやってファイルを読み、書き出すのかについて説明します。

FORTRAN は 1950 年代に開発され、その後時代に合わせて書式や機能が変化してきました。FORTRAN は時代遅れのプログラミング言語と思われがちですが、科学の分野ではまだ広く用いられています。とくに、水文気象学の分野で重要なプログラムである、気候モデル、陸面過程モデル、河川モデルなどは世界的に見ても、FORTRAN で書かれることが多いようです。

なお、このテキストでは FORTRAN を使いますが、自分でプログラミングをするときには、C、C++、C#、Java、Ruby など、好きなプログラミング言語を使ってかまいません。また、この章では FORTRAN のごく限られた、また少々特殊な使い方しか紹介できません。きちんと勉強したい人は、FORTRAN の教科書を読んでください。

#### 5.1 FORTRANソースコードの基本

FORTRAN のソースコードを見てみましょう。これは端末に「Hello, World」などの文字を出力するプログラムです。

```
1:      program hello
2:c
3:c this is comment.
4:c
5:      write(6,*) 'Hello, World'
6:      write(6,*) 'A very very long sentence such as this
7:      $      must be divided in two lines'
8:c
9:      end
```

1 行目では「hello」という名前のプログラムが始まることが宣言されています。「program」というのが開始宣言の命令で、FORTRAN のソースコードの 1 行目に必ず書かなければなりません。FORTRAN のソースコードで本文を書けるのは 7 桁目から 7 2 桁目までです。この例でも 1 行目は行頭からスペースを 6 つ空けて 7 桁目から「program hello」と書き出されています。2 文字目から 5 文字目には「行番号」という特別な意味を持つ整数を入れることができるのですが、現在はほとんど使われません。使い方を知らない人は FORTRAN の教科書を参照してください。

2 行目から 4 行目はコメント文です。コメントを書きたい場合には、1 桁目に c または \*を入力します。ということになります。

5 行目には「Hello, World」と端末に書き出す命令が書かれています。「write」というのが書き出しの命令です。括弧内の最初にあるのは「装置番号」を、次にあるのは「書式」を表します。装置番号とは書き出し先を示す正の整数です。装置番号はユーザが定義するのですが、5 と 6 だけはそれぞれ標準入力（キーボード）、標準出力（端末）と決まって

います。書式が「\*」というのは自動書式という意味です。その後「”」で囲まれた文字列が出力される文字列となります。

6 行目から 7 行目には「A very very long sentence such as this must be divided in two lines」と端末に書き出す命令が書かれています。この場合、72 桁目までに命令を書ききれないので 2 行に分けます。2 行にわたって命令を書きたい場合は、続きの行の 6 桁目に 0 以外の文字を入力します。この例では「\$」を用いています。

## 5.2 FORTRAN ソースコードのコンパイル

それでは練習のために、まずこの例と全く同じテキストファイルを `emacs` で作成し、`example1.f` という名前で保存して下さい。さて、このファイルはこのままでは実行できません。FORTRAN コンパイラという特別なコマンドを利用して、実行可能な機械語に変換する必要があります。この作業を「FORTRAN ソースコードをコンパイルする」といいます。Appendix C に従ってあなたの UNIX コンピュータが設定されている場合、`gfortran` という FORTRAN コンパイラが使えるようになっているはずです。FORTRAN ソースコード `example1.f` をコンパイルするには次のようにします。

```
% gfortran example1.f
```

すると、`a.out` という実行可能ファイルが作成されます。これを

```
% a.out
```

のように実行すると、

```
Hello, World
A very very long sentence such as this must be divided in two lines
```

のように端末に表示されるはずです。ソースコードをコンパイルしてできるファイルの名前は全て `a.out` となります。ただ、これではどのソースコードをコンパイルしたか後で分からなくなります。そこで「-o」というオプションをつけ、その後にファイル名を入れると、実行ファイル名を変えることができます。例えば、

```
% gfortran example1.f -o example1
```

のように、とすると、`example1` という実行ファイルができます。これは `gfortran` 以外の多くの FORTRAN コンパイラで有効なようです。

## 5.3 FORTRAN のファイルの入出力

第 4 章の表 4-2 で紹介したコマンドの一つに `cmap2asc` があります。このプログラムは、バイナリデータをアスキーデータに変換するコマンドでした。例えば、第 4 章で使った 1987 年 1 月のバイナリファイルを `temp.txt` という名前のテキストファイルに変換したい場合は、



```
% cd ~/BINARY
% cmap2asc CMAP203_19870100.cmap temp.txt
```

とするのでしたね。

この cmap2asc のソースコードが `~/unix_semi/src/cmap2asc.f` にあります。

この節では、このプログラムのソースコードを4つのブロックに区切って、解説していきます。

```

1:      program cmap2asc
2:ccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccc
3:cto    Convert binary (144x72) to ascii
4:con    30th/May/2002
5:cby    nhanasaki
6:cat    IIS, UT
7:ccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccc
8:      implicit none
9:c
10:      integer          nx
11:      integer          ny
12:      parameter        (nx=144)
13:      parameter        (ny=72)
14:c
15:      integer          i,j
16:      real              rdat(nx,ny)
17:      character*128     cifname
18:      character*128     cofname
19:c
20:      integer          iargc

```

第1ブロックでは、まず1行目でプログラムの開始を宣言しています。「**program** ○○○」と書くことで、このコードは○○○というプログラムだということが宣言できます。

2行目から7行目にかけてはコメント文で、作成者や日付などが記録されています。

8 行目には「implicit none」と書いてありますが、これは「暗黙の変数型」<sup>26</sup>を使わないことを意味するもので、忘れずに書く習慣を付けてください。

10行目から20行目にかけて記述されているのが変数の型の宣言です。第3章では **Bourne** シェルスクリプトの変数と値の代入について勉強しました。**FORTRAN** でも同じように変数に値を代入することができますが、より細かく設定を行うようになっています。ここでは、変数(第3章で空き箱だといいました)を宣言しています。**integer** は整数型、**real** は実数型、**character** は文字型の宣言に用います。変数の宣言は図 5-1 のように「数字や文字の入る箱の種類や大きさを決めること」と考えるとわかりやすいでしょう。たとえば「**integer nx**」というのは、整数(**integer**)をひとつだけ入れることができる **nx** という箱を用意することです。「**character\*128 cifname**」は 128 文字までの文字列(**character\*128**)を入れることのできる **cifname** という箱を用意すること、といった具合です。**rdat(nx,ny)** というのは **nx** 列 **ny** 行に区切られた実数を格納できる箱だと考えてください。こうした2つ以上の値を代入できる変数を配列(**array**)といいます。

1 2 行目と 1 3 行目にある `parameter` は変数に値を定数として代入します。この値は定数となり、プログラム実行中は書き換えることはできません。ここで `nx` と `ny` は配列 `rdat` の宣言で用いられています。この場合、`nx` と `ny` は必ず定数として値が代入されていなければなりません。

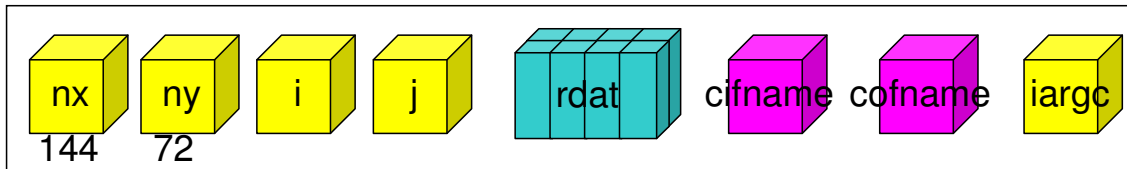


図 5-1 FORTRAN の変数の型

```

1:ccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccc
2:c Get file name
3:ccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccc
4:      if (iargc().ne.2) then
5:        write(6,*) 'Usage: cmap2asc Infile Outfile'
6:        stop
7:      end if
8:c
9:      call getarg(1,cifname)
10:     call getarg(2,cofname)

```

第2ブロックでは、ファイル名を読み込んでいます。

まず4行目ですが、`if` の直後にある括弧内の条件文を判定し、真である場合は次の行にある命令を実行します。条件文の書き方は表 5-1 の通りです。ここで「`iargc()`」はコマンド（この例の場合 `cmap2asc`）に与えられた引数の数を返す関数です。よって、ここでの条件文は「引数の数が2でなければ真」ということになります。真の場合、次の行の命令が実行されます。偽の場合何も実行されず、`end if` 以降の命令を実行します。

5行目の **write** 文は既に勉強しました。括弧の中で装置番号と書式を指定するのですでしたね。装置番号の「6」は端末を表します。書式の「\*」は自動設定を表します。どういう書式で書き出すかは細かく指定できるので、知りたい人は **FORTRAN** の教科書を参照してください。書き出したい文字列は「**'**」の間に書きます。6行目の「**stop**」はプログラムの終了を意味します。つまり、4行目から7行目を普通の言葉で書くと、「もし引数の数が2でなかったら、画面に'Usage: cmap2asc Infile Outfile'とデフォルトの書式で端末に出力してコマンドを終了します。それ以外の場合（つまり引数の数が2なら）、作業を継続します。」となります。

9 行目、10 行目はそれぞれ 1 番目と 2 番目の引数を読み込んでいます。1 番目の引数を読み、変数 `cifname` に代入するのが `call getarg(1,cifname)` という命令です。表 4-2 を見ると `cmap2asc` には入力ファイル名と出力ファイル名の 2 つの引数が必要ですね。9 行目と 10 行目を普通の日本語にすると、「1 番目の引数を変数 `cifname` に代入します。2 番目の引数を変数 `cofname` に代入します。」となります。

表 5-1 条件文の種類

|        |                                                        |
|--------|--------------------------------------------------------|
| a.gt.b | a is <b>greater</b> <b>than</b> b (a が b より大きいとき真)     |
| a.ge.b | a is <b>greater</b> <b>equal</b> than b (a が b 以上のとき真) |
| a.lt.b | a is <b>less</b> <b>than</b> b (a が b より小さいとき真)        |
| a.le.b | a is <b>less</b> <b>equal</b> than b (a が b 以下のとき真)    |
| a.eq.b | a is <b>equal</b> to b (a が b に等しいとき真)                 |





## 宿題

---

1. 表 4-2 にある addcmap の FORTRAN ソースコードを読んで、普通の言葉に直してみましょう。ソースコードは~/unix\_semi/src/addcmap.fにあります。
2. cmap2asc.f と反対の機能を持つプログラム、asc2cmap.f を作りましょう。(編集が終わったら、コンパイルして実行してみましょう)

※宿題の答えは~/unix\_semi/lesson5 にあります。

## 第 6 章

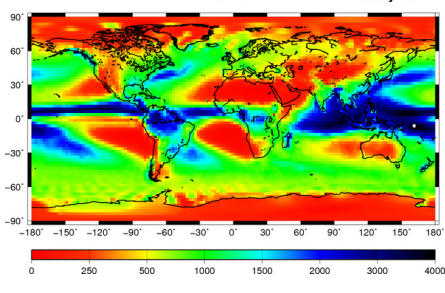
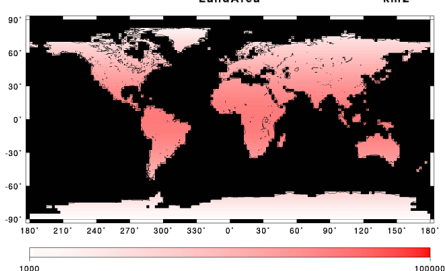
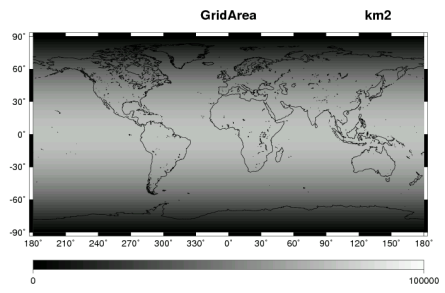
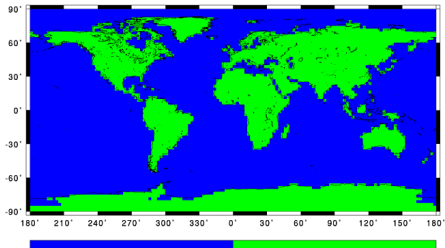
### グリッドデータの処理(2) FORTRAN プログラムを使ったマスク処理

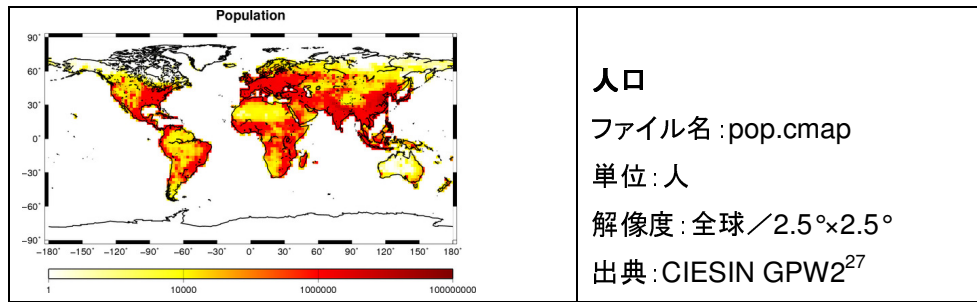
東京の年降水量はどのくらいでしょうか。また、世界の年平均降水量はどのくらいでしょうか。CMAP データを使って調べてみましょう。世界のどのくらいの地域で、年降水量が東京を上回るのでしょうか。また、どのくらいの人が 500mm/year より降水量の少ない場所に住んでいるのでしょうか。

#### 6.1 この章で利用するデータ

この章では表 6-1 のデータを用います。データは~/unix\_semi/lesson6 にあります。

表 6-1 第6章で利用するデータ(バイナリファイル)

|                                                                                     |                                                                                                                                                                        |
|-------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|   | <p><b>年平均降水量(1979-2001)</b><br/>           ファイル名 : prc.cmap<br/>           単位 : mm/year<br/>           解像度 : 全球 / 2.5°×2.5°<br/>           備考 : CMAP データを使って作成したもの</p> |
|  | <p><b>グリッド面積(陸のみ)</b><br/>           ファイル名 : lndara.cmap<br/>           単位 : km<sup>2</sup><br/>           解像度 : 全球 / 2.5°×2.5°</p>                                    |
|  | <p><b>グリッド面積(海陸)</b><br/>           ファイル名 : grdara.cmap<br/>           単位 : km<sup>2</sup><br/>           解像度 : 全球 / 2.5°×2.5°</p>                                     |
|  | <p><b>海陸マップ</b><br/>           ファイル名 : lndmsk.cmap<br/>           単位 : なし<br/>           解像度 : 全球 / 2.5°×2.5°<br/>           備考 : 陸が 1、海が 0</p>                        |



## 6.2 CMAP のバイナリファイル进行操作するためのコマンド

第 4 章では CMAP のバイナリファイル进行操作するためのコマンドについて勉強しましたね。ここではさらに多くのコマンドを説明します (表 6-2)。

表 6-2 利用可能なコマンド (その 2)

| プログラム        | 引数                              | 機能                                                                                       |
|--------------|---------------------------------|------------------------------------------------------------------------------------------|
| avecmap      | IN, miss                        | バイナリ IN の miss 以外のデータの平均値を計算する                                                           |
| maxcmap      | IN, miss                        | バイナリ IN の miss 以外のデータの最大値を見つける                                                           |
| mincmap      | IN, miss                        | バイナリ IN の miss 以外のデータの最小値を見つける                                                           |
| sumcmap      | IN, miss                        | バイナリ IN の miss 以外のデータの総和を計算する                                                            |
| pointcmap    | IN, lon, lat                    | 経度が lon、緯度が lat の場所のデータを表示する                                                             |
| findcmap     | IN, sign, num, OUT              | バイナリ IN のデータで num と sign するものを表示し、バイナリ OUT に書き出す。それ以外は 0 が入る。                            |
| rplocmap     | IN, sign, num1, num2, OUT       | バイナリ IN のデータで num1 と sign するものを num2 に置き換えてバイナリ OUT に書き出す。それ以外は IN と同じデータが入る。            |
| maskcmap     | IN, MASK, sign, num, OUT        | バイナリ MASK のデータが num と sign するとき、バイナリ IN のデータをバイナリ OUT に書き出す。それ以外は 0 が入る。                 |
| maskrplocmap | IN, MASK, sign, num1, num2, OUT | バイナリ MASK のデータが num1 と sign するとき、バイナリ IN のデータを num2 に置き換えて OUT に書き出す。それ以外は IN と同じデータが入る。 |

IN, MASK, OUT はそれぞれバイナリファイル  
sign は条件を示し、eq, ne, gt, ge, lt, le から一つを選択できる。それぞれ「と一致する」、「と一致しない」、「より大きい」、「以上の」、「より小さい」、「以下の」となる。  
 num, num1, num2 は実数、lon, lat は経度と緯度を表す実数  
 miss は実数の欠損値<sup>28</sup>

表 6-2 にある findcmap, rplocmap, maskcmap, maskrplocmap は少しわかりにくいので、例を挙げて説明します。まず、findcmap は条件に合ったデータを抽出するコマンドです。図 6-1(a) にある例を考えましょう。左図のように 1 から 16 までのデータが入っているバ

イナリファイル IN から、8 より大きい (**greater than**) 数字を抽出して結果をバイナリファイル OUT に書き出すには以下のようにします。

```
% findcmap IN gt 8 OUT
```

次に **rplccmap** は条件に合ったデータを別の実数に置き換えるコマンドです。ここで、条件に合わないデータは元の実数がそのまま出力されます。図 6-1(b) にある例のように、1 から 16 までのデータが入っているバイナリファイル IN から、8 より大きい (**gt**) 数字を 1 で置き換えて結果をバイナリファイル OUT に書き出すには以下のようにします。

```
% rplccmap IN gt 8 1 OUT
```

**maskcmap** は別に用意したバイナリファイル **MASK** (図 6-1(c) に示されるものでマスクファイルと呼ばれる) が条件を満たすか判定し、条件を満たす場合はバイナリファイル IN のデータを抽出し、バイナリファイル OUT に書き出すコマンドです。図 6-1(d) にある例のように、1 から 16 までのデータが入っているバイナリファイル IN から、マスクファイル **MASK** が 1 と一致する(**eq**)ときだけデータを抽出してバイナリファイル OUT に書き出すには以下のようにします。

```
% maskcmap IN MASK eq 1 OUT
```

最後に **maskrplccmap** は別に用意したバイナリファイル **MASK** が条件を満たすか判定し、条件を満たす場合はバイナリファイル IN のデータを別の実数に置き換えて、バイナリファイル OUT に書き出すコマンドです。図 6-1(e) にある例のように、1 から 16 までのデータが入っているバイナリファイル IN から、マスクファイル **MASK** が 1 と一致する(**eq**)ときだけデータを 1 に変換して、結果をバイナリファイル OUT に書き出すには以下のようにします。

```
% maskrplccmap IN MASK eq 1 1 OUT
```

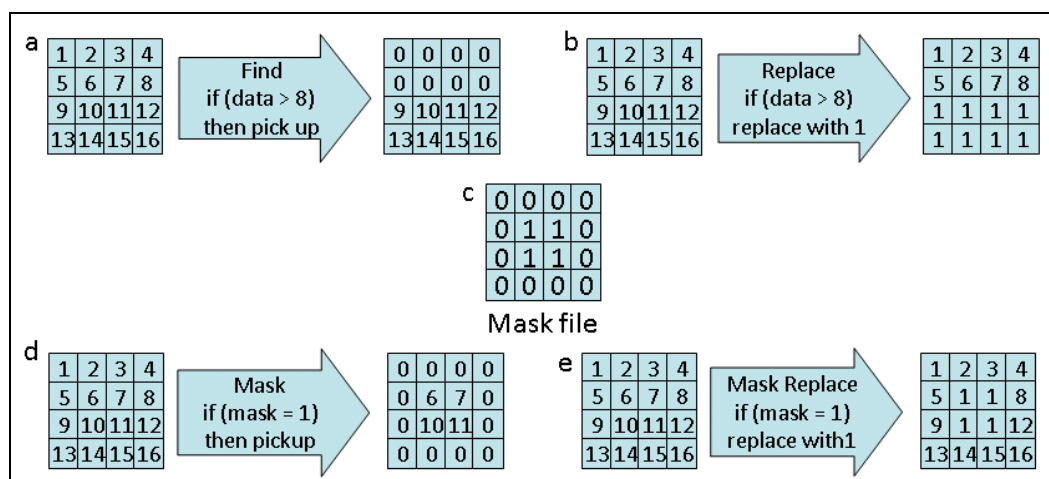


図 6-1 findcmap, rplccmap, maskcmap, maskrplccmap の概念

#### 【例 1】

東京（東経 138.75° 北緯 36.25°<sup>29)</sup>）での年降水量を求めましょう。



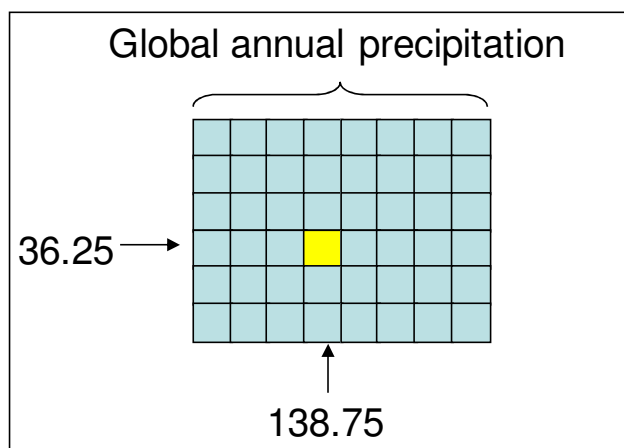


図 6-2 pointcmap の概念

まず表 6-1 にあるどのデータを使えばよいでしょう？また表 6-2 にあるコマンドのどれを使えばよいでしょう？年平均降水量データ(prcp.cmap)を pointcmap で処理するのがよさそうですね。以下のようにコマンドを打ち込みましょう。

```
% pointcmap prcp.cmap 138.75 36.25
```

このようにすると、図 6-2 に示すように、バイナリファイル prcp.cmap を読み取り、東京付近のグリッド(北緯 36.25, 東経 138.75)のデータを取り出すことができます。興味のある人は、~/unix\_semi/src/pointcmap.f を見て、どのようなソースコードになっているのか調べてみてください。

### 【例 2】

世界の年平均降水量を求めましょう。

次に、世界の年平均降水量を求めましょう。表 6-1 にあるどのデータと表 6-2 にあるどのコマンドを使えばよいでしょう？年平均降水量データ(prcp.cmap)を avecmap で処理するのがよさそうですね。以下のようにコマンドを打ち込みましょう。

```
% avecmap prcp.cmap -999
```

このようにすると、バイナリファイル prcp.cmap の全てのデータを足しあわせ、データの数で割って平均値を計算することができます。このとき欠損値（上の例では-999）はデータの足し合わせから除外されます。実はこの計算は正確ではありません。このことについては、この章の宿題で考えてみましょう。

### 【例 3】

世界のどのくらいの地域で、年降水量が東京を上回るのでしょうか。面積を求めましょう。

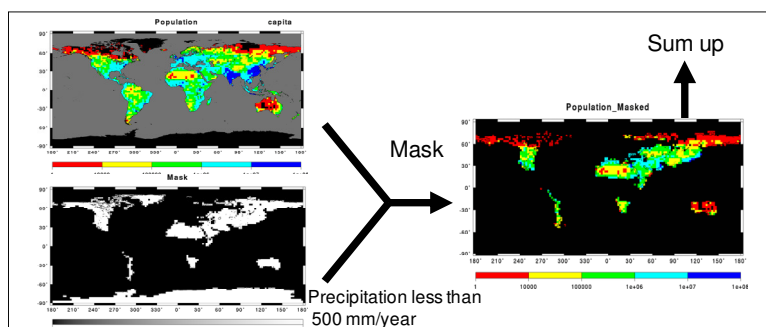


図 6-3 例4の計算方法

東京の年降水量は例 1 で求めたように 1530mm/year でしたね。そこでまず陸域の面積データ(lndara.cmap)を、年降水量データ(prcp.cmap)が 1530mm/year 以上の場合だけ抜き出しましょう。

```
% maskcmap lndara.cmap prcp.cmap ge 1530 temp.cmap
```

次に、抽出した面積データの合計を求めましょう。

```
% sumcmap temp.cmap 0
```

#### 【例 4】

年降水量が 500mm/year より少ない地域に住んでいる人口を求めましょう。

例 3 と同じように、人口データ (pop.cmap)を、年降水量データ(prcp.cmap)が 500mm/year 未満の場合だけ抜き出しましょう (図 6-3)。

```
% maskcmap pop.cmap prcp.cmap lt 500 temp.cmap
```

抜き出したグリッドの人口の合計を求めましょう。

```
% sumcmap temp.cmap 0
```

## 宿題

1. 世界の年平均降水量を正しく計算しましょう。例 1 では全てのデータの単純な平均を取る `avecmap` コマンドを利用して年平均降水量を求めました。しかし、地球は球体ですので、高緯度ほど各グリッドの面積が小さくなっています (表 6-1 の面積データを見ましょう)。そこで、面積で重みづけした平均値を求めましょう。まずどのようにデータを処理すればよいのか考えましょう。つぎに、表 6-2 にあるコマンドを組み合わせ処理を実行しましょう。ヒントは表 6-1 にある地表面面積データ (`grdara.cmap`) と表 4-2 にあったバイナリファイル同士の掛け算を行う `procmap` をうまく使うことです。なお、途中である数値をある数値で割り算しないといけませんが、ここは電卓などを使って手計算を行ってください。
2. 陸上の年平均降水量を次の表のように分類した場合、それぞれの地域はどれくらいの面積になるのでしょうか。また、それぞれの地域にどれくらいの人が住んでいるのでしょうか。次の表を埋めてみましょう。

| Precipitation (mm/year) | Population (capita) | Area (km2) |
|-------------------------|---------------------|------------|
| 0-50                    |                     |            |
| 50-100                  |                     |            |
| 100-500                 |                     |            |
| 500-1000                |                     |            |
| 1000-2000               |                     |            |
| 2000-                   |                     |            |
| Total                   |                     |            |

※宿題の答えは `~/unix_semi/lesson6` にあります。

## 第 7 章

### FORTRAN (2) FORTRAN の時系列データの入出力

第 7 章では、時系列のバイナリファイルを効果的に処理するための FORTRAN プログラム技術について説明します。なお第 5 章でも述べましたが、基本的な FORTRAN の文法については Appendix D にある参考文献で自習を続けてください。また、この章で紹介するソースコードは ~ / unix\_semi / src / l にあるので、必要に応じて参照してください。

#### 7.1 FORTRAN の時系列ファイルの入出力

第 4 章では月単位の CMAP バイナリファイルを年単位に変換することを学びました。この処理を行うため、mon2yearcmap というコマンドを利用しました。mon2yearcmap コマンドは引数として与えられた開始年から終了年までの各年の 1 月から 12 月のデータを読み、各年の平均値データを出力します。このとき、月の日数の違いを考慮するため、月毎のデータを足し合わせるとき、各月のデータに各月の日数をかけておき、最後に 1 年の日数 (365 日、うるう年なら 366 日) で割る計算を行います。このソースコードが ~ / unix\_semi / src / mon2yearcmap.f にあるので開いてみましょう。この章ではこのソースコードを 4 つのブロックに分けて解説していきます。

```

1:      program mon2yearcmap
2:      ccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccc
3:      implicit none
4:      c parameter
5:      integer          nx, ny
6:      parameter        (nx=144, ny=72)
7:      c variable
8:      integer          ix, iy
9:      integer          iyear, iyearmin, iyearmax, imon
10:     c temporary
11:     character*128     ctmp
12:     c function
13:     integer           iargc
14:     integer           igetday
15:     integer           len_trim
16:     c in
17:     real              rmondats(nx,ny) !! Monthly data array
18:     c out
19:     real              ranudats(nx,ny) !! Annual data array
20:     c local
21:     integer           inofdaymon
22:     integer           inofdayyear
23:     c files
24:     character*128     cdir, cexp, crun, csuf
25:     character*128     cifname, cofname
26:     integer           ilen
27:     character*128     clen

```

まず、第 1 ブロックでは変数の宣言をしています<sup>30</sup>。13 行目から 22 行目までの「!!」以降の文字列はコメント文です。文章の途中からコメントを入れるこのやり方は標準的な FORTRAN の文法にはありませんが、たいていどのコンパイラでも使えるようです。便利なので積極的に使い、コードを分かりやすくしましょう。

```

1: cccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccc
2: c Get Arguments
3: cccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccc
4:     if(iargc().ne.6)then
5:         write(*,*) 'mon2year dir exp run yearmin yearmax suf'
6:         stop
7:     end if
8: c
9:     call getarg(1,cdir)
10:    call getarg(2,corg)
11:    call getarg(3,crun)
12:    call getarg(4,ctmp)
13:    read(ctmp,*) iyearmin
14:    call getarg(4,ctmp)
15:    read(ctmp,*) iyearmax
16:    call getarg(5,csuf)
17: c
18:     ilen=len_trim(cdir)
19:     write(clen,*) ilen

```

第2ブロックでは引数を読み込んでいます。このプログラムは6つの引数が必要です。引数を代入する変数ですが、ディレクトリ名、プロジェクト名、実験名、拡張子を示す変数 `cdir`, `corg`, `crun`, `csuf` は文字列型なのですが、開始年と終了年を示す変数 `iyearmin`, `iyearmax` は整数型です。FORTRAN の文法の制約上、引数には文字列型しかとれません。そこで、開始年と終了年はいったん `call getarg` を使って `ctmp` に文字列として読み込んだ後、`read` 文を使って `ctmp` という文字列型変数を整数型変数の `iyearmin` と `iyearmax` に読み込むという作業を行っています(13行目と15行目)。`read` 文や `write` 文の基本的な使い方は第5章で学んだように、`write`(装置番号, 書式)あるいは `read`(装置番号, 書式)と使うのでしたね<sup>31</sup>。ここでは、装置番号に変数名が書かれていますが、これは変数を読んだり、書いたりすることを示します。

18行目と19行目では変数 `cdir` に代入された値の文字数を数えています。これはこのプログラムの後半で利用します。18行目では `len_trim` という1次元の配列を変数 `ilen` に代入しているように見えますが、これは `len_trim` という関数(function)に `cdir` という引数を与えていることを示します。関数とは、コマンドの中におくコマンドだと考えてください。関数はこの例のように、一つの値（整数、実数、文字列）のようにふるまいます。この関数は引数の文字数を計算するもので、ソースコードは `~/unix_semi/src/len_trim.f` にあります。例えば「`cdir='/home/naota'`」の場合、`len_trim(cdir)=11` となります。19行目では整数型変数 `ilen` を文字列型変数 `clen` に書き込むことにより、変数の型を変換しています。先ほどの例の場合、整数 11（じゅういち）が文字列 11（いちいち）に変換されることになります。

```

1: cccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccc
2: c Sum up
3: cccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccc
4: do iyear=iyearmin,iyearmax
5: c clear variable
6:   inofdayyear=0
7: c clear array
8:   do iy=1,ny
9:     do ix=1,nx
10:      ranumat(ix,iy)=0.0
11:    end do
12:  end do
13: c
14:   do imon=1,12
15:     inofdaymon=igetday(iyear,imon)
16:     inofdayyear=inofdayyear+inofdaymon
17: c read data
18:   write(cifname,'(a'//clen//',' ,a4,a4,i4.4,i2.2,i2.2,a5)' )
19:   $   cdir,corg,crun,iyear,imon,0,csuf
20:   call read_binmat(rmondats,nx,ny,15,cifname)
21: c calculation
22:   do iy=1,ny
23:     do ix=1,nx
24:       ranumat(ix,iy)
25:       $   =ranumat(ix,iy)+rmondats(ix,iy)*real(inofdaymon)
26:     end do
27:   end do
28: end do

```

← 配列と変数に入っている値を  
消去する

第3ブロックと第4ブロックでは、do ループを使って繰り返し計算をしています。4行目の「do iyear=iyearmin,iyearmax」に対応する「end do」は第4ブロックの12行目にあります。つまり、この間の処理を繰り返すことになります。

最初の破線で囲まれた部分は後で説明することにして、14 行目の `do imon = 1, 12` の行を見てください。ここでは、変数 `iyear` を値 `iyearmin` から値 `yearmax` までループさせる最中に、さらに変数 `imon` を値 1 から値 12 までループさせています。これによって、`iyearmin` 年 1 月から `yearmax` 年 12 月まで順々に処理を行うことができます。

15 行目は `igetday` という 2 次元の配列を読んでいるように見えますが、これは `igetday` という関数(function)に `iyear` と `imon` という 2 つの引数を与えて呼んでいることを示します。この関数は西暦〇年〇月が何日あるかを計算するもので、ソースコードは `~/unix_semi/src/igetday.f` にあります。この関数はうるう年の 2 月の処理をきちんとこなすので、例えば `igetday(1988,2)` なら 29 となります。

18 行目から 19 行目にかけては **write** 文がありますね。この **write** 文の装置番号は文字列型変数 **cifname** となっています。これは結果を **cifname** に書き込むことを示すのでしたね。続いて **write** 文の書式ですが、これまでは自動設定を表す「\*」を使ってきましたが、ここでは手動設定がされています。まず、シングルクォーテーションの内側にある括弧で囲まれた文字列が書式を示しています。つまり、「a'//clen//,a4,a4,i4.4,i2.2,i2.2,a5」が書式を表しています。結論から言うとこれは「**clen** 文字の文字列、4 文字の文字列、4 文字の文字列、4 桁の整数、2 桁の整数、2 桁の整数、5 文字の文字列を書き出す」ということを表します。まず、最初にある「a'//clen//」は非常に変わった書き方ですが、「長さが **clen** の文字列」を表します<sup>32</sup>。例えば **clen** が 4 なら 4 文字、8 なら 8 文字となります。「a4」は文字列(ascii)が 4 つ、「i4.4」とあるのは 4 桁の整数で必ず 4 桁の整数で埋めること<sup>33</sup>を示します。「a4.a4」のように間にあるカンマ(,)は見やすさのためにあるので無視されます。こ

のように `write` 文を使って書式を持った文字列を変数に代入しています（この場合、変数 `cifname` に `CMAP203_19870100.cmap` などと代入している）。

20 行目は `call` 文の後に `read_binmat` という 5 次元の配列が書かれているように見えますが、これは 5 つの引数を持つ `read_binmat` というサブルーチンを呼び出すことを意味します。サブルーチンとは、コマンドの中におくコマンドだと考えてください。このサブルーチンは「 $n_x \times n_y$  の配列の `cifname` というファイル名のバイナリファイルを装置番号 15 で読み込み、結果を `rmondatt` に代入する」機能を持ちます。詳細については次の節で説明します。

24 行目から 25 行目は変数 `ranudatt` に各月のデータを足し込む作業をしています。

28 行目には `do imon = 1, 12` に対応する `end do` があり、ループが終了します。

最後に、説明を飛ばしていた一つめの破線で囲まれた箇所の解説をしましょう。`do` ループを使って繰り返し処理を行ったとき、配列と変数には前の値が入っています。例えば 1987 年から 1988 年までの計算を行うとき、まず変数 `ranudatt` には 1987 年の合計値が入りますが、何もしなければ 1988 年になっても残ってしまい、1987 年と 1988 年の合計を足すような処理になってしまうのです。そこで、各年の最初の足し込みを行う前に、配列と変数に入っている値を消去します。これは数字を足し合わせていくような処理では、とても重要な作業です。

```

1: c Get average
2:   write(*,*) 'Total num of days:', ndayyear
3:   do iy=1,ny
4:     do ix=1,nx
5:       ranudatt(ix,iy)=ranudatt(ix,iy)/ndayyear
6:     end do
7:   end do
8: c Write results
9:   write(cfname, '(a'//cflen//',a4,a4,i4.4,i2.2,i2.2,a5)')
10:  & corg,crun,iyear,0,0,csuf
11:   call wrte_binmat(ranudatt,nx,ny,16,cfname)
12: end do
13: c
14: end

```

第 4 ブロックでは、2 行目から 7 行目までで配列の平均値を求めています。このコマンドの処理を図で表すと、図 7-1 のようになります。その後 9 行目から 11 行目までで結果を書き出しています。ここでは、「`read_binmat`」とちょうど逆の操作をする「`wrtb_binmat`」というサブルーチンを呼んでいます。このサブルーチンは「`cfname` という名前のバイナリファイルを装置番号 16 で開き、 $n_x \times n_y$  の配列 `ranudatt` を書き込む」という機能を持ちます。詳細については次の節で説明します。

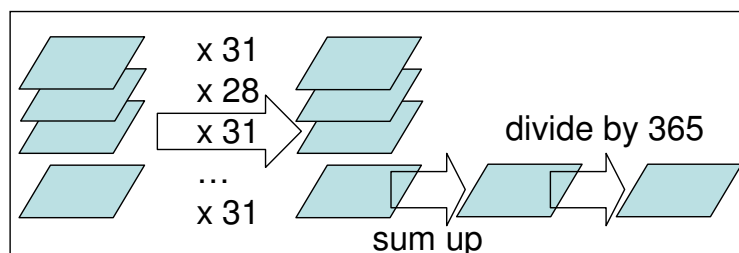


図 7-1 mon2yearcmap の計算手順

## 7.2 FORTRAN の Subroutine

mon2yearcmap.f では read\_binmat と wrte\_binmat の 2 つのサブルーチンが用いられていました。この節では、サブルーチンについて解説します。

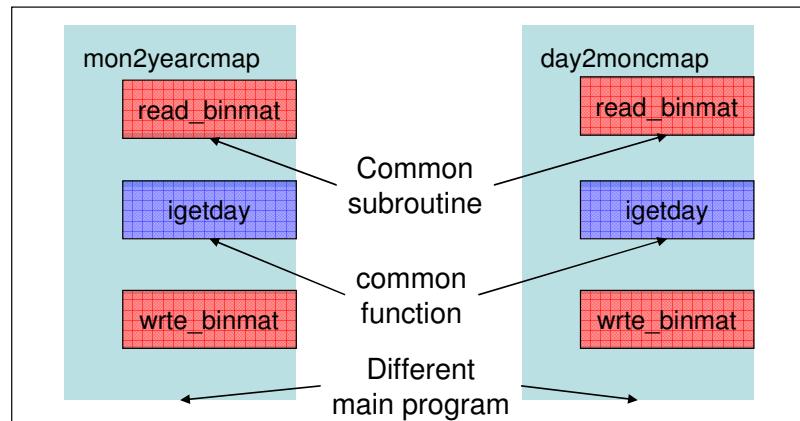


図 7-2 プログラム、サブルーチン、関数の関係

mon2yearcmap は月平均データを読み込み、年平均データを出力するコマンドですが、同様に日平均データを読み込み、月平均データを出力する day2moncmap というコマンドを作りたいとします。この 2 つのコマンドでは、ファイルを読み込んだり、書き出したりするタイミングは異なりますが、ファイルを読み込んだり、書き出したり、ある月の日数を計算したりする作業は共通して発生します。このとき、図 7-2 のように、mon2yearcmap と day2moncmap という別の主プログラムで、共通のサブルーチンと関数を使うことができます。このようにいくつかのプログラムで同じ処理を行いたい時にサブルーチンは効果的です。

まず、プログラムでサブルーチンを呼ぶ時には、

**call サブルーチン名 (引数 1, 引数 2, 引数 3, ...)**

という形を使います。引数はサブルーチンに受け渡され、サブルーチンの中の処理が終わるとプログラムに返されます。

ではサブルーチン read\_binmat.f のソースコードを見てみましょう。

```

1: subroutine read_binmat (rdat,nx,ny,infile,cfname)
2: ccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccc
3:   open (infile,file=cfname,access='DIRECT',status='old',recl=4*nx)
4:   do j = 1,ny
5:     read (infile,rec=j) (rdat(i,j),i=1,nx)
6:   end do
7:   close (infile)
8: c
9:   end

```

まず 1 行目で、サブルーチンであることを宣言しています。かっこの中は引数で、プログラムから受け渡されます。引数は、書いてある順番通りに受け渡されるので、プログラ



ムとサブルーチンで変数の名前が違っていてもかまいませんが、変数の型や配列の大きさは厳密に一致していなければいけません。たとえば `mon2yearcmap` の場合、プログラムの変数 `rmondatt` の値がサブルーチンの変数 `rdatt` に受け渡されることになります。その他の 3 行目から 7 行目までは第 5 章で学んだ構文と全く同じであることが分かります。このように「ファイルを開いて中に記述された数値を配列に代入する」という機能を抜き出したものをサブルーチンと呼びます。

## 7.3 make と Makefile

`mon2yearcmap` のようにプログラムとサブルーチン、関数が別々のファイルに用意されている場合、ひとつのプログラムに統合する作業が必要です。これを管理するためのプログラムが `make` コマンド、`make` コマンドに指示を与えるファイルが **Makefile** です。

`mon2yearcmap` をコンパイルするときにはまず `mon2yearcmap.f` というプログラムのソースコードと、`read_binmat.f` `wrte_binmat.f` というサブルーチンのソースコード、`igetday.f`, `len_trim.f` という関数のソースコードをそれぞれ **FORTRAN** コンパイラでコンパイルし、オブジェクトファイルという機械語に翻訳します。これらのオブジェクトファイルはそれぞれ、`mon2yearcmap.o`, `read_binmat.o`, `wrte_binamt.o`, `igetday.o`, `len_trim.o` という名前になります。次にこれらの 5 つのオブジェクトファイルを組み合わせ、`mon2yearcmap` という一つのコマンドにします。

`mon2yearcmap` の **Makefile** は `~/unix_semi/src/Makefile` にあります。この **Makefile** は `mon2yearcmap` をコンパイルする以外にも様々な設定が記述されているのですが、説明を簡単にするため、以下では `mon2yearcmap` に関するものだけ説明します。なお、このテキストでは **Makefile** の細かい文法について説明できません。詳しく知りたい人は、Appendix D にある参考文献などを参照してください。

```

1:#####
2:#      Macro
3:#####
4:RM      = /bin/rm          # path to command rm
5:FC      = gfortran        # path to fortran compiler
6:FFLAGS  =                  # option for fortran compiler
7:#####
8:#      Suffix rule
9:#####
10:_.f.o:
11:  _Tab_ $(FC) $(FFLAGS) -c $<
12:#####
13:#      Dependency
14:#####
15:TARGET1 = mon2yearcmap
16:COMPONENT1 = read_binmat.o wrte_binmat.o \
17:             igetday.o len_trim.o
18:#####
19:#      Compilation
20:#####
21:$(TARGET1) : $(TARGET1).o $(COMPONENT1)
22:_Tab_ $(FC) -o $@ $@.o $(COMPONENT1)

```

**Makefile** は `make` の文法に沿って記述されたスクリプトです。この文法は様々な点で Bourne シェルスクリプトと異なりますが、構造は似ています。

1 行目から 3 行目まではコメント文です。**Makefile** でも「#」以降はコメントです。4 行目から 6 行目は変数に値を代入しています。Bourne シェルスクリプトの場合は「=」の

前後にスペースを入れてはいけませんが、**Makefile** の場合は入れてもかまいません。16 行目のように途中で改行するときは、行末に\ (バックスラッシュ) をつけ直後に改行を入れます。

1 0 行目から 1 1 行目にかけては拡張子「.o」のオブジェクトファイルをどのように作るのが示されています。文法については説明しませんが、拡張子「.f」の **FORTTRAN** ソースコードを変数 **FC** に代入された **FORTTRAN** コンパイラを使ってコンパイルすることによってつくる、ということが示されています。

1 5 行目から 1 7 行目にかけては、ファイルの依存関係についての変数を定義し、その値を代入しています。今作ろうとしているファイルは実行ファイル **mon2yearcmap** で、これを作るには、自分自身のオブジェクトファイル **mon2yearcmap.o** 以外に、**read\_binmat.o**、**wrte\_binmat.o**、**igetday.o**、**len\_trim.o** の 4 つのオブジェクトファイルが必要です。このことを **mon2yearcmap** はこれら 4 つのオブジェクトファイルに依存しているといえます。この例では、作ろうとしているコマンドを代入する変数を **TARGET1**、依存しているオブジェクトファイルを代入する変数を **COMPONENT1** としています。

2 1 行目から 2 2 行目にかけては、コンパイルの命令をしています。まず 2 1 行目はファイルの依存関係を示します。変数に値を代入すると次のようになります。

```
mon2yearcmap : mon2yearcmap.o read_binmat.o wrte_binmat.o igetday.o len_trim.o
```

「:」(コロン) 記号は、その左側にあるファイルがその右側にあるファイルに依存していることを示します。つまり、右側にあるファイルのうち、左側にあるファイルより新しいものはコンパイルをやり直すことになります。2 2 行目はコンパイルの仕方が示されています。変数に値を代入すると次のようになります。

```
gfortran -o mon2yearcmap mon2yearcmap.o read_binmat.o wrte_binmat.o igetday.o len_trim.o
```

最初の「**gfortran -o mon2yearcmap**」は第 5 章で学んだように、実行ファイルを「a.out」という名前ではなく「**mon2yearcmap**」という名前にすることを示すのでしたね。そのあとにオブジェクトファイル名が引数として続いています。これらは **FORTTRAN** コンパイラによって<sup>34</sup>一つの実行ファイルに結合されます。

なお、破線で囲まれた部分は<tab>です。<space>ではエラーになるので注意してください。**Make** コマンドが実行されると同じディレクトリにある **Makefile** を探しますので、別のディレクトリにある **Makefile** は無視されます。

## 宿題

- 年と月を入力するとその月の日数を出力するプログラムを組みましょう。ファイルの名前は「program」としてください。コンパイルして、実行してみましょう。次のようになったでしょうか。

```
% program 1995 3
31
% program 1995 2
28
% program 1996 2
29
% program 2000 2
29
% program 1900 2
28
```

※宿題の答えは~/unix\_semi/lesson7 にあります。

## 第 8 章

### Bourne シェルスクリプト(2) Do ループ、For ループと If 文

第6章の宿題2では、最低 42 個のコマンドを端末に打ち込む必要がありました。同じようなコマンドを何度も入力するのは手間がかかる上、間違いをしやすいので避けたいところです。このような作業を減らすためにはどうすればよいのでしょうか。

#### 8.1 シェルスクリプトの制御文と条件文

第 1 章や第 3 章で説明したように、UNIX ではテキストファイルにコマンドを保存し、実行権限を与えると、処理を実行できるのでしたね。この時に Bourne シェル文法に沿って記述されたファイルを Bourne シェルスクリプトといいます。Bourne シェル文法にはループや if 文などの制御文があり、これらを活用するとより高度な処理ができます。Bourne シェルスクリプトで使える主な制御文を表 8-1 に、条件文を表 8-2 にまとめました。

表 8-1 Bourne シェルスクリプトで使える主な制御文

|                                                            |                                                                                                       |
|------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| For ループ                                                    | <b>for VAR in VALUES; do<br/>process<br/>done</b>                                                     |
| While ループ                                                  | <b>while [ 条件文 ]; do<br/>process<br/>done</b>                                                         |
| IF 文                                                       | <b>if [ 条件文 ]; then<br/>process1<br/>elif [ 条件文 ]; then<br/>process2<br/>else<br/>process3<br/>fi</b> |
| VAR: 変数名<br>VALUES: 値のリスト<br>[ 条件文 ]: 条件文<br>process: コマンド |                                                                                                       |

表 8-2 Bourne シェルスクリプトで使える条件文

|                              |                                      |
|------------------------------|--------------------------------------|
| FILENAME という<br>ファイルがあるかの判定  | <b>[ -f FILENAME ]</b> <sup>35</sup> |
| DIRNAME という<br>ディレクトリがあるかの判定 | <b>[ -d DIRNAME ]</b>                |
| A=B(文字列を判定する場合)              | <b>[ A = B ]</b>                     |
| A≠B(文字列を判定する場合)              | <b>[ A != B ]</b>                    |

|                 |             |
|-----------------|-------------|
| A=B (整数を判定する場合) | [ A -eq B ] |
| A≠B 整数を判定する場合   | [ A -ne B ] |
| A > B           | [ A -gt B ] |
| A ≥ B           | [ A -ge B ] |
| A < B           | [ A -lt B ] |
| A ≤ B           | [ A -le B ] |
| And             | -a          |
| Or              | -o          |

## 8.2 【例1】For ループを使った Bourne シェルスクリプト

1 月から 12 月までをあらわす 1 から 12 までの数字を端末に表示し、誕生月（8 月だとします）の場合に「Happy birth month!」と出力する Bourne シェルスクリプトを書いてみましょう。

```

1: #!/bin/sh
2: #####
3: MONS="01 02 03 04 05 06 07 08 09 10 11 12"
4: BIRTHMON="08"
5: #####
6: for MON in $MONS; do
7:   if [ $MON = $BIRTHMON ]; then
8:     echo $MON "Happy birth month!"
9:   else
10:    echo $MON
11:   fi
12: done

```

1 行目の `#!/bin/sh` は Bourne シェルスクリプトを宣言しています。3 行目と 4 行目は、「MONS」「BIRTHMON」という変数それぞれに値を代入しています。6 行目は「for ループ」という繰り返しの制御文です。ここにある「for MON in \$MONS; do」は「変数 MON にリスト MONS にある値を順番に代入して、do と done の間にあるコマンドを繰り返す」ことを示します。この場合では MON=01, MON=02,... MON=12 と MON に MONS にある 12 の値を次々に代入することになります。7 行目は「if 文」という条件判定に基づく処理を行います。ここにある「if [ \$MON = \$BIRTHMON ]; then」というのは、もし変数 MON が変数 BIRTHMON と一致するならば」という条件を表します。この条件を満たす場合は 8 行目が実行されます。「echo」は文字列を端末に出力するコマンドです。条件を満たさない場合は 9 行目にある「else」の次の行である 10 行目が実行されます。

## 8.3 【例2】While ループを使ったシェルスクリプト

1 月 1 日から 12 月 31 日までを端末に表示し、誕生日（2 月 28 日だとします）には「Happy birth day!」と出力する Bourne シェルスクリプトを作りましょう。

```

1: #!/bin/sh
2: #####
3: YEAR=2005
4: MONS="01 02 03 04 05 06 07 08 09 10 11 12"
5: BIRTHMON="02"
6: BIRTHDAY="28"
7: #####
8: for MON in $MONS; do
9:     DAY=1                # refresh day
10:    DAYMAX=`nofday $YEAR $MON` # set the last day of the month
11:    while [ $DAY -le $DAYMAX ]; do
12:        if [ $MON = $BIRTHMON -a $DAY = $BIRTHDAY ]; then
13:            echo $MON $DAY "Happy birth day!"
14:        else
15:            echo $MON $DAY
16:        fi
17:        DAY=`expr $DAY + 1`      # day for next iteration
18:    done
19: done

```

この Bourne シェルスクリプトは例 1 を拡張して作られています。9 行目から 18 行目が大きく変わっていますね。8 行目の For ループにより、変数 MON には 1 から 12 までの値が代入されます。9 行目では変数 DAY に 1 を代入しています。その後 10 行目では、〇年〇月の日数を調べるためのコマンド「nofday」を使っています（表 4-2）。これは第 7 章の宿題で皆さんが作ったものとも同じはずです。例えば「nofday 2005 01」の結果は 31 となります。ここで記号「`」（バッククォーテーション）は「バッククォーテーションで囲まれたコマンドの実行結果」を示します。つまり、YEAR=2005、MON=01 の場合、まずバッククォーテーションの中身が 31 と計算され、その結果が DAYMAX=31 というように変数に代入されるわけです。11 行目には「while ループ」という繰り返し処理があらわれます。ここで「while [ \$DAY -le \$DAYMAX ]; do」は「変数 DAY の値が変数 DAYMAX の値以下のとき、do から done までの間にあるコマンドを繰り返す」ことを示します。この do に対応する done は 18 行目にありますね。その直前の 17 行目に「DAY=`expr \$DAY + 1`」という表現があるでしょう。expr コマンドは四則演算を行います。この場合、変数 DAY に 1 を足し、その結果を変数 DAY に代入する、という操作を表します。このとき、\$DAY, +, 1 のそれぞれの間にスペースが必要ですから注意しましょう。

## 8.4 【例3】第6章の宿題2を一つの Bourne シェルスクリプトで処理する

第 6 章の宿題 2 では、最低 42 回コマンドを端末に打ち込む必要がありました。この処理を一つのシェルスクリプトにまとめてみましょう。まずは第 6 章の宿題 2 で端末に打ち込んだコマンドをテキストファイルに書いてみましょう。なぜなら、このファイルに実行権限を与えるだけでシェルスクリプトになるからです。例えば、年間降水量が 0mm/year 以上 50mm/year 未満、50mm/year 以上 100mm/year 未満の地域に住む人口を求めるシェルスクリプトは次のようになるでしょう。

```

1: # 0mm/year =< Precipitation < 50mm/year
2: maskcmap pop.cmap          rain.cmap ge 0 temp.pop.ge0.cmap
3: maskcmap temp.pop.ge0.cmap rain.cmap lt 50 temp.pop.ge0.lt50.cmap
4: sumcmap temp.pop.ge0.lt50.cmap 0
5: # 50mm/year =< Precipitation < 100mm/year
6: maskcmap pop.cmap          rain.cmap ge 50 temp.pop.ge50.cmap
7: maskcmap temp.pop.ge50.cmap rain.cmap lt 100 temp.pop.ge50.lt100.cmap
8: sumcmap temp.pop.ge50.lt100.cmap 0

```

それではこのシェルスクリプトから共通の処理を見つけて、値を変数に置き換えてみましょう。たとえば、2行目から4行目と、6行目から8行目は上限値（50 か 100 か）と下限値（0 か 50 か）が異なるだけで、作業は同じですね。そこで、上限値を変数 UL (upper limit)、下限値を変数 LL (lower limit)として、シェルスクリプトを書き換えてみましょう。また宿題2では人口と面積の2つについて処理をする必要がありましたから、popを変数 DAT (data)としてやはり書き変えてみましょう。すると、次のように書き直せるはずです。こうすると、3行目から5行目と、8行目から10行目は全く同じになりますね。

```

1: # 0mm/year =< Precipitation < 50mm/year
2: DAT=pop; LL=0; UL=50
3: maskcmap ${DAT}.cmap          rain.cmap ge ${LL} temp.${DAT}.ge${LL}.cmap
4: maskcmap temp.${DAT}.ge${LL}.cmap rain.cmap lt ${UL} temp.${DAT}.ge${LL}.lt${UL}.cmap
5: sumcmap temp.${DAT}.ge${LL}.lt${UL}.cmap 0
6: # 50mm/year =< Precipitation < 100mm/year
7: DAT=pop; LL=50; UL=100
8: maskcmap ${DAT}.cmap          rain.cmap ge ${LL} temp.${DAT}.ge${LL}.cmap
9: maskcmap temp.${DAT}.ge${LL}.cmap rain.cmap lt ${UL} temp.${DAT}.ge${LL}.lt${UL}.cmap
10: sumcmap temp.${DAT}.ge${LL}.lt${UL}.cmap 0

```

次に、do ループと for ループ、if 文を組み合わせて、シェルスクリプトを完成させましょう。まず変数 JOB を定義し、LL=0, UL=50 を JOB=1、LL=50, UL=100 を JOB=2 としましょう。次に while ループを使って、この変数 JOB を 1 から 2 まで繰り返しましょう。同じように変数 DAT を定義し、for ループを使って pop から lndara まで繰り返しましょう。すると、以下のようになります。

```

1: #!/bin/sh
2: #####
3: for DAT in pop lndara; do
4:   JOB=1
5:   while [ $JOB -le 2 ]; do
6:     if [ $JOB -eq 1 ]; then
7:       LL=0; UL=50
8:     elif [ $JOB -eq 2 ]; then
9:       LL=50; UL=100
10:    fi
11:    maskcmap ${DAT}.cmap          rain.cmap ge ${LL} temp.${DAT}.ge${LL}.cmap
12:    maskcmap temp.${DAT}.ge${LL}.cmap rain.cmap lt ${UL} temp.${DAT}.ge${LL}.lt${UL}.cmap
13:    sumcmap temp.${DAT}.ge${LL}.lt${UL}.cmap 0
14:    JOB=`expr $JOB + 1`
15:  done
16: done

```

変数と制御文を利用してまとめることで、スクリプトをすっきりと書くことができましたね。

## 宿題

1. CMAP のサイトからダウンロードできるオリジナルデータ（アスキーファイル）が `~/unix_semi/dat_org` にあります。第 2 章の宿題で利用した技術を使って、これらのアスキーファイルを全て 144x72x4 byte の月別のバイナリファイルに変換するシェルスクリプトを書きましょう。このとき、バイナリファイルの名前が `~/unix_semi/dat_bin` にあるものと同じになるようにしましょう。

ヒント

桁を揃えて出力するには

```
% YR=2
% YR=`echo $YR|awk '{printf("%2.2d", $1)}'`
```

というように入力しましょう。つまり、1 行目で変数 `YR` に 2 という数字を代入し、2 行目でいったんこれを `echo` コマンドで表示し、パイプを使って `awk` コマンドに渡し、`awk` の書式付プリントコマンド `printf` で 2 桁のうち 2 桁を必ず使う整数表記（2.2d）を指定します。詳しい文法が知りたい人は、インターネット検索して下さい。

また、西暦の下 2 桁を変数とした場合、使用するデータが 1999 年から 2000 年に変わる時にどのように対処すれば良いか考えましょう。例えば、1999 年までの処理と 2000 年以降の処理とを分けて 2 つのループ文を作ると良いでしょう。または、変数が 3 桁になったら変数から 100 をひいて 2 桁に戻すという方法も有効でしょう。

※宿題の答えは `~/unix_semi/lesson8` にあります。



## Appendix A

### Xming のインストールと使い方

Windows コンピュータから UNIX コンピュータにログインするには PC X サーバというソフトウェアが必要です。このテキストで紹介するのは無償で提供されている Xming というソフトウェアです。このソフトをインストールし、基本的な使い方を覚えましょう。

#### A.1 Xming のインストール

1. Xming のサイト<sup>36</sup>に行く。
2. “distributed”のリンクをクリックする。



図 A-1 手順 2 のスクリーンショット

3. “SourceForge Project Xming”のリンクをクリックする。

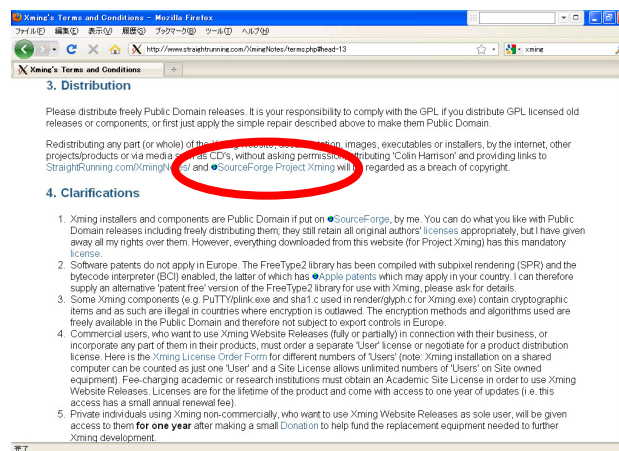


図 A-2 手順 3 のスクリーンショット

4. “View all files”のボタンをクリック。

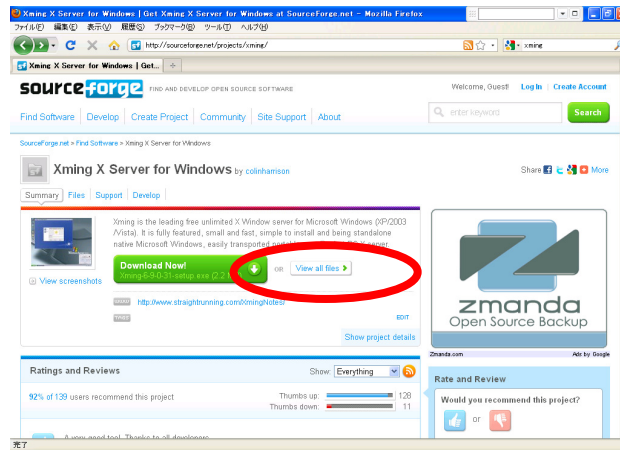


図 A-3 手順 4 のスクリーンショット

5. “Xming-fonts”をクリックし、現れた“7.5.0.11”（あるいは類似のもの）をクリックする。同様に、“Xming”をクリックし、現れた“6.9.0.31”（あるいは類似のもの）をクリックする。“Xming-fonts-7-5-0-11-setup.exe”と“Xming-6-9-0-31-setup.exe”がブラウザに表示されていることを確認する。

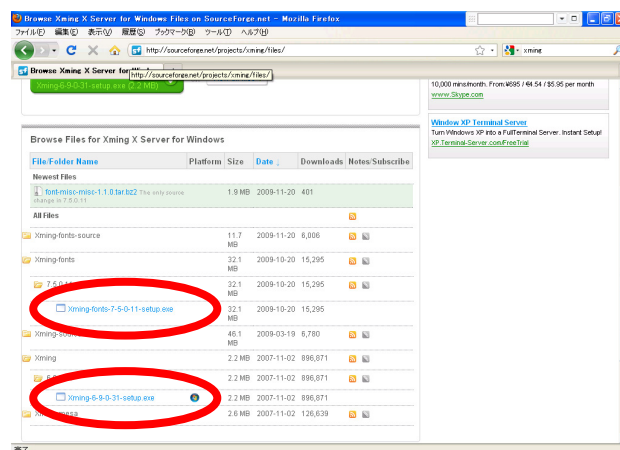


図 A-4 手順 5 のスクリーンショット

6. “Xming-fonts-7-5-0-11-setup.exe”と“Xming-6-9-0-31-setup.exe”がダウンロードされたことを確認する。
7. “Xming-fonts-7-5-0-11-setup.exe”を実行する<sup>37</sup>。基本的に、インストールが終了するまで「次へ」をクリックし続けて問題ない。



図 A-5 手順 7 のスクリーンショット

8. “Xming-6-9-0-31-setup.exe”を実行する。基本的に、インストールが終了するまで「次へ」をクリックし続けて問題ない。



図 A-6 手順 8 のスクリーンショット

## A.2 Xming の使い方

1. Xming を起動する。スタートメニューから “Xlaunch”アイコンを探してクリックする。

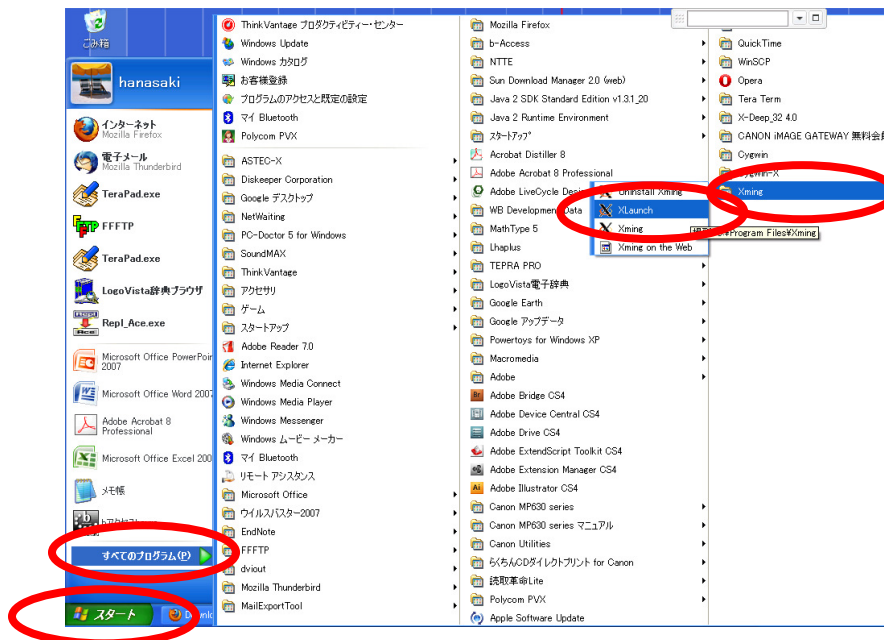


図 A-7 使い方の手順 1 のスクリーンショット

2. “Multiple windows”を選択する。

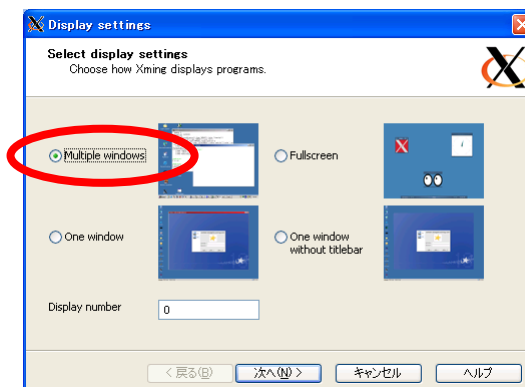


図 A-8 使い方の手順 2 のスクリーンショット

3. “Start a program”を選択して次へ。

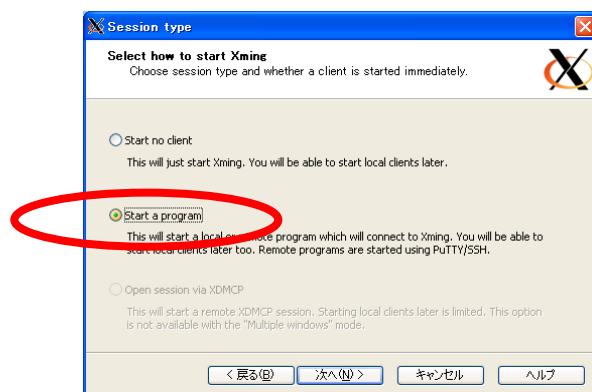


図 A-9 使い方の手順3のスクリーンショット

4. “Using PuTTY”を選択肢、“Connect to computer”の欄に接続する UNIX コンピュータ名を入力。後の2つの欄は空欄のままでよい。

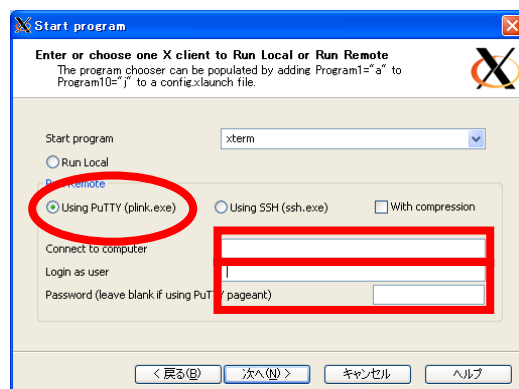


図 A-10 使い方の手順4のスクリーンショット

5. もう 1 枚ウィンドウがあらわれるが、何も選択せずに次へ。

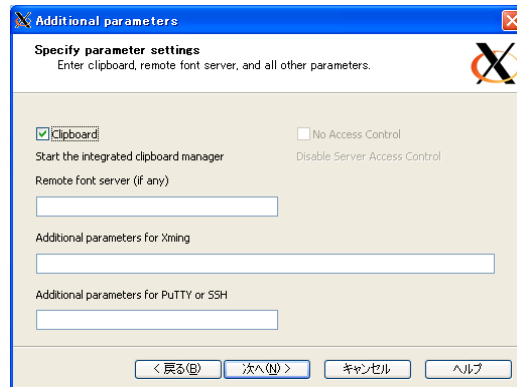


図 A-11 使い方の手順5のスクリーンショット

6. この選択を保存したいときは “Save configuration” をクリックする。次回以降、手順 1 ～6 を省略できる。最後に「完了」 ボタンをクリックする。

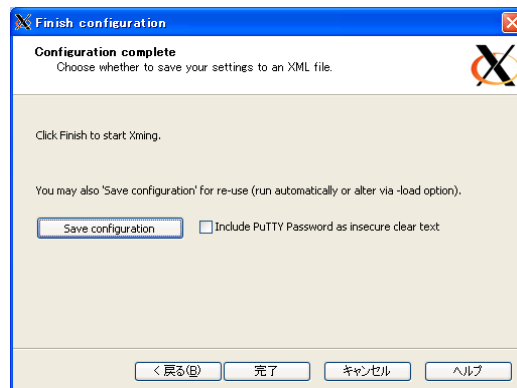


図 A-12 使い方の手順6のスクリーンショット

7. 最初に接続するときには次のようなウィンドウがあらわれる。「はい」 を選択。



図 A-13 使い方の手順 7 のスクリーンショット

8. ID（ログイン名）を入力する。



図 A-14 使い方の手順 8 のスクリーンショット

9. パスワードを入力する。



図 A-15 使い方の手順9のスクリーンショット

10. このような画面が現れたら接続に成功です。



図 A-16 使い方の手順 10 のスクリーンショット

### A.3 困った時に

うまくいかないときは一度 Xming を終了し、XLaunch を立ち上げるところから再開してください。このとき、タスクバーにある X のアイコンを探し、右クリックして終了 (Exit) してください。

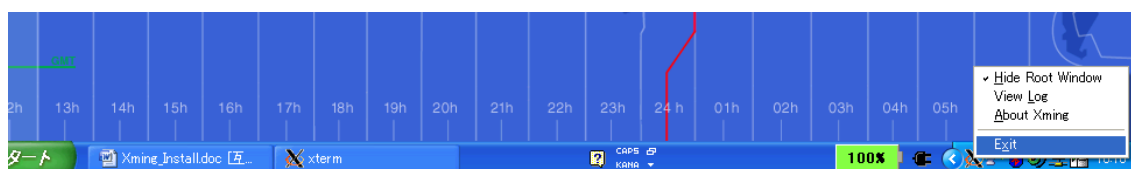


図 A-17 Xming の Exit の仕方

うまくログインができないときは、ログ（履歴）を見るとヒントが見つかることがあります。同じくタスクバーにある X のアイコンを右クリックして View Log をクリックして下さい。

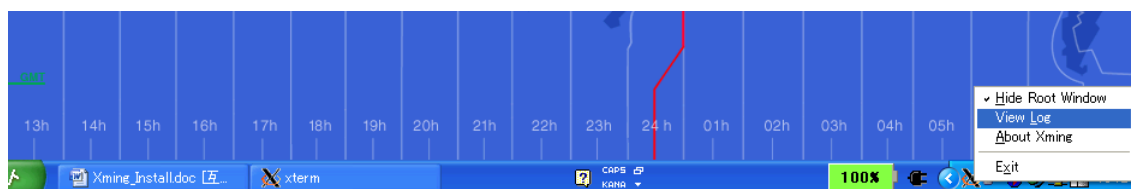


図 A-18 Xming の Log の見方

## Appendix B

### UNIX コンピュータの設定(管理者権限が不要な事項)

この章ではサーバである UNIX コンピュータの設定について説明します。この章に書いてあることを実施するのに管理者権限は必要ありませんが、基本的な UNIX 操作の知識が必要です。このテキストを使って初めて UNIX を勉強する方は管理者の方に頼んで、代わりに設定してもらうようにして下さい。

#### B.1 テキスト教材の導入

このテキストを利用するユーザのホームディレクトリに移動し、テキストの教材をダウンロードしましょう。なお、各コマンドの役割については、本テキスト p2 表 1-1 などを参考にして下さい。まず、ウェブブラウザを起動します。

```
% cd ~
% firefox
```

1. <https://sites.google.com/site/naotahanasaki/home> にアクセスします。
2. マニュアルに移動します。
3. 教材をダウンロードします。

次に、tar アーカイブを伸張しましょう。

```
% gunzip unix_semi.tar.gz
% tar xf unix_semi.tar
```

こうすると、unix\_semi\_20120601 のようなディレクトリができたはずです。ディレクトリ名が長いので次のようにして **unix\_semi** という名前に変更しましょう。

```
% mv unix_semi_20120601 unix_semi
```

次に環境変数を設定します。ホームディレクトリにある **.bashrc** を編集します。

```
% emacs ~/.bashrc
```

まず、**~/unix\_semi/src** にパスを通します。これによって **src** 内にあるサンプルコマンドがどこのディレクトリからも使えるようになります。次に環境変数を変更して **gfortran** で入出力するバイナリファイルのエンディアンを **big** に設定します<sup>38</sup> (ただし、装置番号 25-26 番は **Little** とします)。Emacs の操作方法については本テキスト p3 表 1-2 を参照してください。

```

1:#####
2:# PATH                                ←コメント文: タイトル
3:#####
4:PATH=.:${PATH}:~/unix_semi/src        ←パスの設定
5:#####
6:# FORTRAN                            ←コメント文: タイトル
7:#####
8:export GFORTRAN_CONVERT_UNIT="big_endian;little_endian:25-26"    ←設定

```

編集が終わったら、設定を有効にします。次のコマンドを実行して下さい。

```
% source ~/.bashrc
```

次にサンプルコマンドをコンパイルします。ディレクトリ `src` にはサンプルコマンドのプログラム文が保存されており、サンプルコマンドをコンパイルすることでそれらのコマンドが使用可能になります。

```

% cd ~/unix_semi/src
% make veryclean
% make all

```

これで準備完了です。

## B.2 Intel Fortran Compiler を使う場合

もし `gfortran` でなく、Intel Fortran Compiler を利用する場合は次のようにして下さい。

1. 入出力を Big Endian で記述するために、`~/.bashrc` に次のように加えます。ただし、装置番号 25-26 番は Little とします。  
`export F_UFMTENDIAN="big;little:25,26"`
2. 次に、`write` 文を使ったバイナリ出力で指定する `recl` 値の単位を `byte` に変えるため<sup>39</sup>、`~/.bashrc` に次のように加えます。  
`alias ifort='ifort -assume byterecl'`
3. また `~/unix_semi/src/Makefile` を次のように書き変えます。  
10 行目 `FC = ifort -assume byterecl`  
12 行目 `FL = ifort -assume byterecl`
4. `~/unix_semi/src` にあるコードを再コンパイルします。

```

% cd ~/unix_semi/src
% make veryclean
% make all

```

## B.3 Little Endian で入出力する場合

もし Big Endian でなく、Little Endian でバイナリを記述したい場合は次のようにして下さい。特に 2009 年 12 月現在、Ubuntu (Linux OS の一つ) が導入された Core 2 Duo 搭載のマシンに `apt-get` を使って `gfortran` をインストールした場合、環境変数



GFORTRAN\_CONVERT\_UNIT="big\_endian"を設定すると gfortran コマンドが正常に実行できない問題があるようです。このような場合は、教材の中にある **Big endian** で記述された全てのバイナリファイルを **Little endian** に書き直す必要があります。

まず、`~/bashrc` にある環境変数 `GFORTRAN_CONVERT_UNIT` の設定を削除します（冒頭に`#`を加えることで、この行をコメント文にすることができます。元に戻すときは`#`を削除することでこの行を有効にします。）。そして、`~/unix_semi/src` にあるプログラムを全て再コンパイルします。

```
% cd ~/unix_semi/src
% make veryclean
% make all
```

次に`~/unix_semi/dat_bin`にあるファイルを変換するため、以下のようにします。

```
% cd ~/unix_semi/lesson8
% sh exercise.sh
```

最後に`~/unix_semi/lesson6`の教材を変換するため以下のようにします。

```
% cd ~/unix_semi/lesson6
% sh asc2bin.sh
```

## Appendix C

### UNIX コンピュータの設定(管理者権限が必要な事項)

この章ではサーバである UNIX コンピュータの設定について説明します。この章に書いてあることを実施するには基本的に管理者権限が必要です。管理者の方に頼んで設定してもらうようにして下さい。

#### C.1 テキスト実施にあたって必要な UNIX 環境

このテキストを実施するには、表 C-1 の環境が必要です。計算機管理者の方は、テキストを実施される方のために以下の環境をご提供ください。

表 C-1 このテキストを実施するために必要な環境

|         |                                                                                                                             |
|---------|-----------------------------------------------------------------------------------------------------------------------------|
| OS      | UNIX あるいは UNIX 系 OS であること。                                                                                                  |
| ログインシェル | 任意ですが、このテキストでは bash を前提に説明をしています。                                                                                           |
| 通信      | ssh によるリモートログインが有効であること。<br>また X11Forwarding が有効であること。                                                                      |
| ソフトウェア  | 以下のソフトウェアが利用可能なこと<br>make<br>gcc (GNU C コンパイラ)<br>GMT (Generic Mapping Tool)<br>imagemagick<br>gfortran (GNU FORTRAN コンパイラ) |

#### C.2 UNIX 環境構築の事例 (Mac OS 10.6 の場合)

以下は、2009 年 11 月に筆者が Mac mini Server (Mac OS 10.6 Snow Leopard Server)に上記の環境を設定した際の手順です。ご参考までに。

- ssh を設定する
  - /etc/sshd\_config ファイルを編集し、「X11Forwarding yes」にする。この設定をしておかないと画像等が Windows コンピュータに転送されない。
  - 同じファイルを編集し、「PasswordAuthentication yes」にする。この設定をしておかないと、Xming 等の PC X サーバソフトウェアでのログインで、パスワード認証エラーになる。
- アカウントを作成する (Mac OS X の場合)
  - Workgroup Manager を起動する。起動には root password が必要。
  - 右上の鍵のアイコンをクリックし、再度パスワードを入力するとユーザ追加が可能になる (このときユーザ名は root にすること)。

- ツールバーにある「新規ユーザ」をクリックするとユーザが追加できる。
  - ここで「基本」タブを埋めただけだとホームディレクトリが作られないので「ホーム」タブでホームディレクトリを設定する<sup>40</sup>。
  - その他のタブは空欄でよい。最後に「今すぐホームを作成」のアイコンをクリックし、保存する。
3. その他の設定 (Mac OS X 特有の問題か)
- 初期設定のままだと、Xming 等の PC X サーバソフトウェアからログインしようとしても、「bash: xterm: command not found」と表示されて接続できない。  
/etc/paths ファイルを編集してパスに/usr/X11/bin を加えると動作するはずなのだが、うまくいかないのが、それぞれのホームディレクトリに.bashrc ファイルを作成し、PATH=\$PATH:/usr/X11/bin という 1 行を加える。
  - 初期設定のままでは Ctrl-Space というキーボード操作に Spotlight の起動が割り当てられている。これは emacs 使用時に困るので、「システム環境設定」にある「キーボード・マウス」のアイコンの「ショートカット」タブで Spotlight のショートカット割当を変更する。
  - バックスラッシュ(\)が入力できないときは、Option(Alt)-¥と打つと出力される。
  - Spotlight を停止するには/etc/hostconfig に SPOTLIGHT=-NO-を追加する。
4. make と gcc のインストール (Mac OS X の場合)
- 初期設定のままでは make や gcc が入っていないので、Apple のサイト<sup>41</sup>から、まず Xcode をダウンロードする。ダウンロードするには ADC membership が必要。ダウンロードしたら、Xcode.mpkg をダブルクリックしてインストール。これで make や gcc がインストールされる。
5. fink のインストール (Mac OS X 版の apt-get ユーティリティのこと)
- fink のサイト<sup>42</sup>に行く。2009 年 11 月現在、Mac OS10.6(Snow Leopard)にはバイナリインストールができないので、ソースからインストールする。インストール時には、設定項目が多数あるが、基本的には全てデフォルトでよい。
  - FinkCommander (0.5.5)をインストール。しかし初期設定のままだと動作しない。環境設定>パス>Perl で Perl のパスを/usr/bin/perl から/usr/bin/perl5.8.9 に変更すると動く。
6. GMT と imagemagick のインストール
- Fink を使ってインストールする。ただし、imagemagick は別の方法<sup>43</sup>の方がインストールにかかる時間が短い。
7. gfortran をインストール
- gfortran のサイト<sup>44</sup>から Mac OS10.5 (Leopard)用のバイナリをダウンロード。これは Mac OS10.6 (Snow Leopard)でも問題なく動作する。

## 謝辞

### 第1版（英語の PowerPoint）の謝辞

このテキストは 2002 年から 2005 年までの夏学期に東京大学生産技術研究所沖研究室で行われた初心者向けの UNIX 講座の補助資料として作られました。プログラミングや UNIX の知識が全くない学生が沖研究室で研究を始めるために最低限必要な知識をまとめて紹介しています。

私の拙い UNIX 講座に参加してくれた右の学生さんに感謝します。山田朋人さん、須賀可人さん、A. Aslam さん、C. Manusthiparom さん、C. Apirumanekul さん、岡澤毅さん、小久保武さん、高垣佳永子さん、斎田渉さん、久保賢一さん、小岩祐樹さん、石崎安洋さん、Q. Tang さん、N.S. Farzin さん、J. Cho さん、M. Lin さん、犬塚俊之さん、咲村隆人さん、碓大輔さん、新井裕子さん、内海信人さん。

沖研究室は地球規模の水循環に関する研究を行っています。その際に膨大な時系列グリッドデータの操作が必要なため、このテキストはここに重点を置いた内容になっています。あえて内容を偏らせることで研究にすぐに役立つように工夫しました。

このテキストは<なんとか>英語で作りました。沖研究室に毎年やってくる熱意ある留学生に言葉のバリアをはらないためです。テキストの英語は文法・用法上の誤りが多々あるかもしれませんし、下手な英語で内容が分かりにくくなっているかもしれませんが、どうぞご勘弁ください。

2005 年 7 月 5 日

花崎直太

### 第2版（日本語の Word）の謝辞

これまで筆者の UNIX 講座の補助資料は英語で作成されていました。これは留学生の多い沖研究室で言葉のバリアを作らないための筆者なりの配慮でしたが、日本人の学生さんからはコンピュータ用語が英語で分かりにくいという苦情が寄せられていました。また、補助資料は筆者が講義をする際の Visual Aid として使いやすいように MS Powerpoint で作成されていました。このため、文章が少なく、復習したり独学したりするのに使いにくいという苦情もありました。そこで、すばらしい翻訳者・編集者の新田友子さんの協力を得て、2007 年 6 月から 7 月にかけて英語の MS Powerpoint スライドを日本語の文章によるテキストに再編集する作業を行いました。筆者が十分な時間をかけられなかったため、説明が不足していたり、論理が飛躍していたりするかもしれません。このテキストを読まれた方は、是非そうしたコメントを筆者までお寄せください。少しずつ改訂していきます。

2007 年 7 月 31 日

花崎直太